



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**PROYECTO "APLICACIÓN MÓVIL COLABORATIVA PARA MEJORAR
LA EFECTIVIDAD EN LA ASISTENCIA DE MASCOTAS PERDIDAS
CON PARTICIPACIÓN DE VOLUNTARIOS EN LA CIUDAD DE TACNA,
2026"**

Curso: Construcción de software II

Docente: RICARDO EDUARDO VALCARCEL ALVARADO

Integrantes:

Chite Quispe Brian Danilo

(2021070015)

Anampa Pancca David Jordan

(2022074268)

Tacna – Perú

2026



Índice

Contenido

Introducción	3
Guía de Laboratorio N° 01 – REVISION DE CODIGO	4
1. Información sobre el evento práctico	4
1.1. Objetivos.....	4
1.2. Equipos, materiales, programas y recursos	4
1.3. Seguridad.....	5
1. Desarrollo de la Práctica.....	5
2. Conclusiones.....	70

Introducción

La calidad del código constituye un aspecto fundamental en el desarrollo de software, debido a que influye directamente en la mantenibilidad, legibilidad, escalabilidad y confiabilidad de una aplicación. A medida que un proyecto crece, la ausencia de estándares y buenas prácticas puede generar problemas como duplicación de código, dificultad para realizar modificaciones, errores recurrentes y una mayor complejidad en el mantenimiento del sistema.

En el presente laboratorio se realiza una revisión y refactorización del código desarrollado para el proyecto “Aplicación móvil colaborativa para mejorar la efectividad en la asistencia de mascotas perdidas con participación de voluntarios en la ciudad de Tacna, 2026”, desarrollado en el curso de Construcción de Software II. La revisión se enfoca principalmente en los módulos relacionados con la autenticación de usuarios, el registro y visualización de avistamientos, y la gestión de comentarios en las publicaciones.

Para esta evaluación se consideran principios y buenas prácticas de construcción de software, tales como evitar la duplicación de código (DRY), utilizar comentarios únicamente cuando aporten información relevante, aplicar el principio Fail Fast, evitar números mágicos, emplear nombres descriptivos, reducir el uso de variables globales, retornar resultados en lugar de utilizar impresiones por consola y organizar adecuadamente los espacios en blanco. Asimismo, se revisan aspectos relacionados con las convenciones de nomenclatura y la responsabilidad de las variables y métodos.

A partir de los problemas identificados en el código original, se realizaron diferentes acciones de refactorización, entre ellas la creación de utilidades reutilizables, la centralización de procesos repetitivos, la incorporación de constantes descriptivas, la validación temprana de datos y archivos, la mejora de los nombres de variables y la reorganización del acceso a los datos almacenados en Firebase. Estas modificaciones buscan obtener un código más limpio, comprensible y fácil de mantener.

Finalmente, la práctica permite comprobar la importancia de realizar revisiones periódicas del código durante el desarrollo de un proyecto, ya que estas permiten detectar deficiencias antes de que se conviertan en problemas mayores y contribuyen a construir una aplicación más ordenada, estable y preparada para futuras mejoras.

Guía de Laboratorio N° 02

1. Información sobre el evento práctico

1.1. Objetivos:

Objetivo general

- Realizar la revisión del código fuente de una aplicación de software, considerando buenas prácticas y estándares de construcción de software, con la finalidad de identificar errores, mejorar la calidad del código y facilitar su mantenimiento.

Objetivos específicos

- Identificar errores y posibles problemas en el código fuente.
- Verificar el cumplimiento de estándares y convenciones de programación.
- Evaluar la estructura, organización y legibilidad del código.
- Detectar código duplicado, innecesario o difícil de mantener.
- Comprobar el correcto uso de nombres para variables, métodos, clases y componentes.
- Proponer mejoras que permitan obtener un código más limpio, seguro y mantenible.
- Documentar las observaciones y recomendaciones encontradas durante la revisión.

1.2. Equipos, materiales, programas y recursos

- Computadora o laptop.
- Conexión a Internet, de ser necesaria.
- Software ofimático para la elaboración de la documentación.
- Software requerido para su aplicación
- Aplicación diseñada e implementada en el Curso de Construcción de Software II
- Formatos de despliegue para la documentación del Proyecto de Software en proceso de implementación

1.3. Seguridad

- Protocolo de Seguridad para laboratorios y talleres
- Protocolo ante la presencialidad en Laboratorios

1. Desarrollo de la Práctica

Caso de Uso 01: "Autenticar Usuario"

Figura 06: Diagrama de Casos de Autenticar Usuario fase análisis

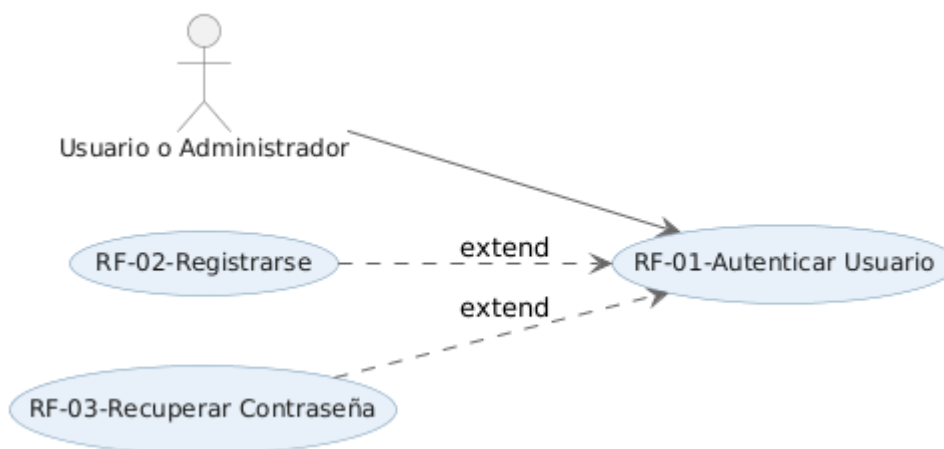


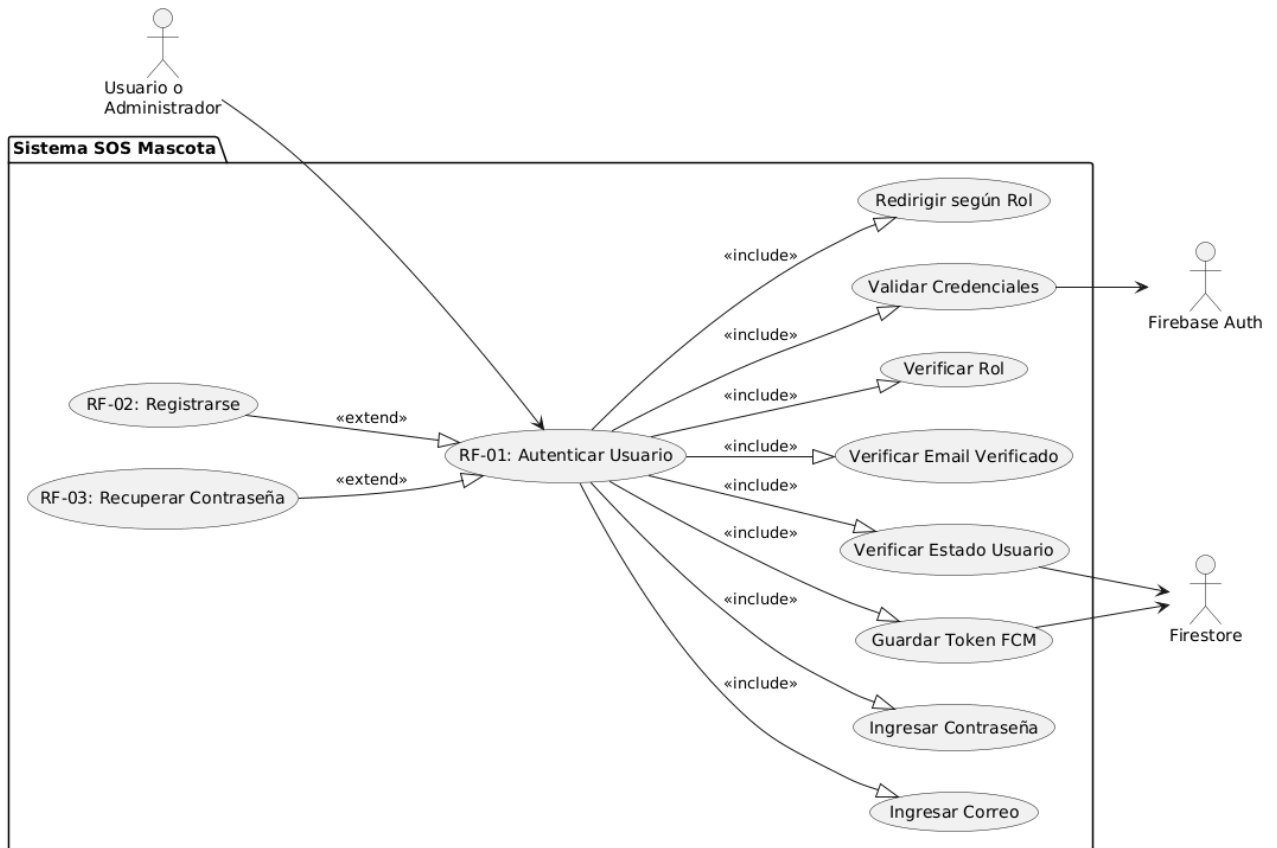
Figura 22: Diagrama de Caso de uso de Autenticar Usuario fase diseño

Figura 22: Diagrama de Secuencia de “Autenticar Usuario” fase análisis

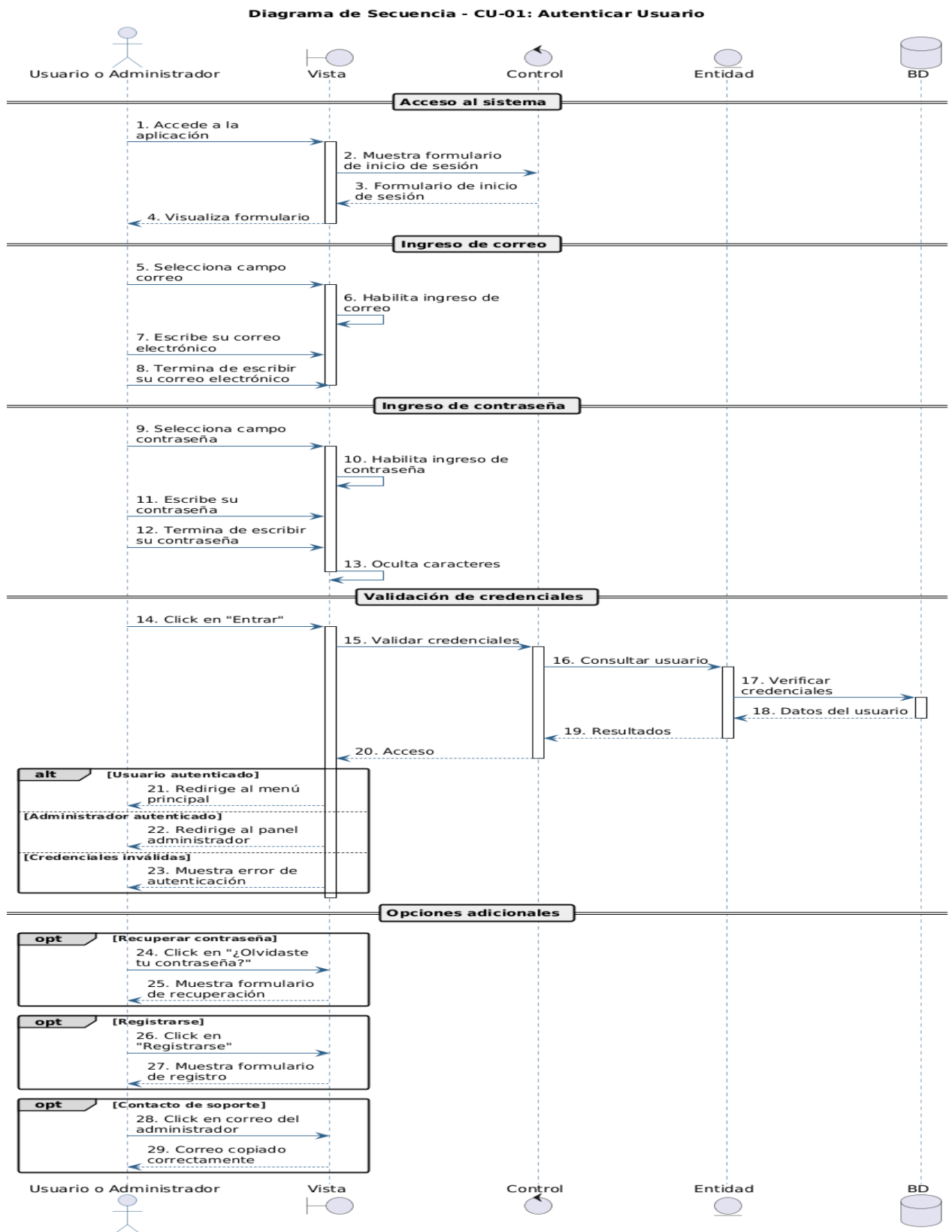


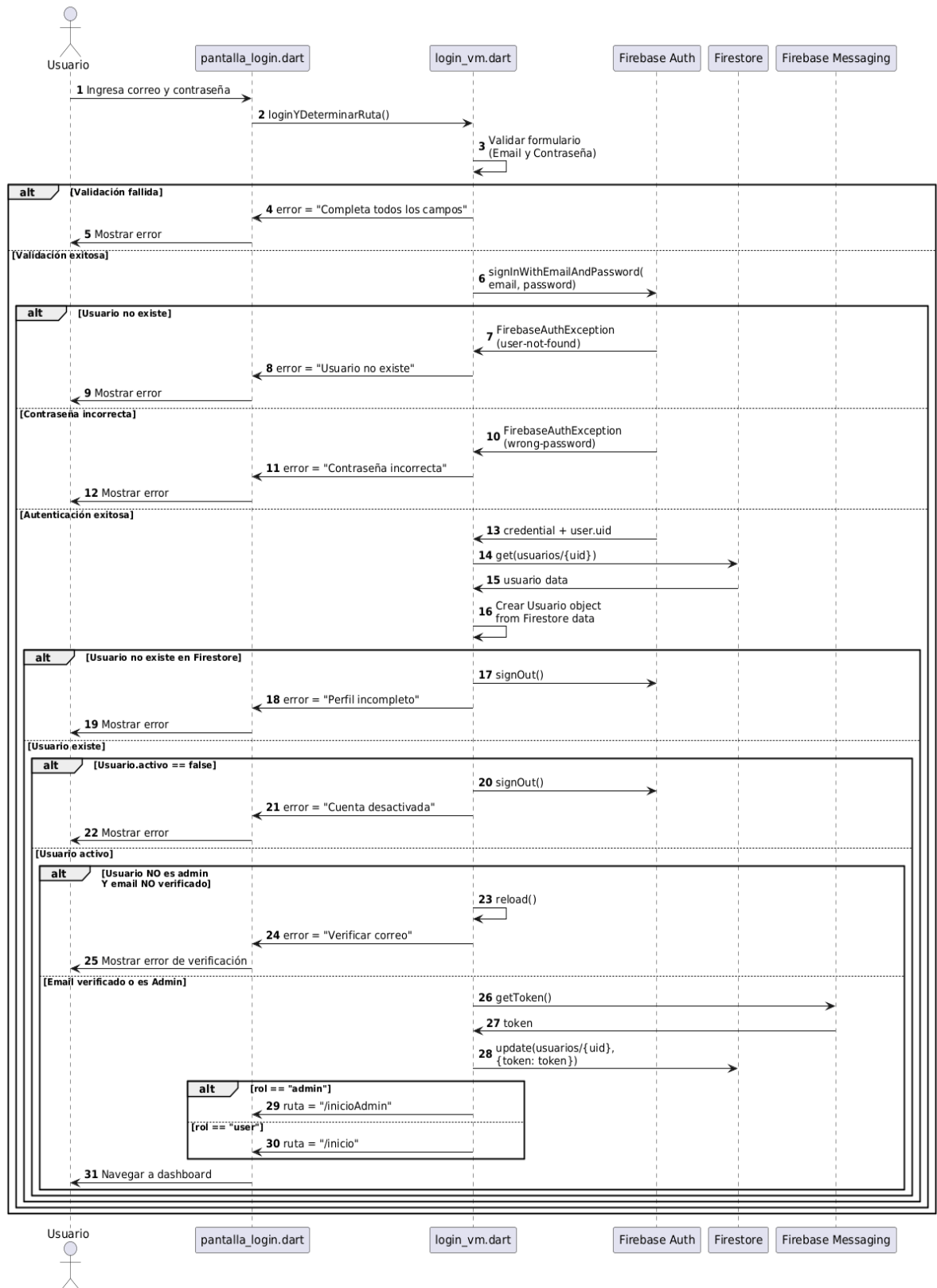
Figura 03: Diagrama de Secuencia de Autenticar Usuario fase diseño


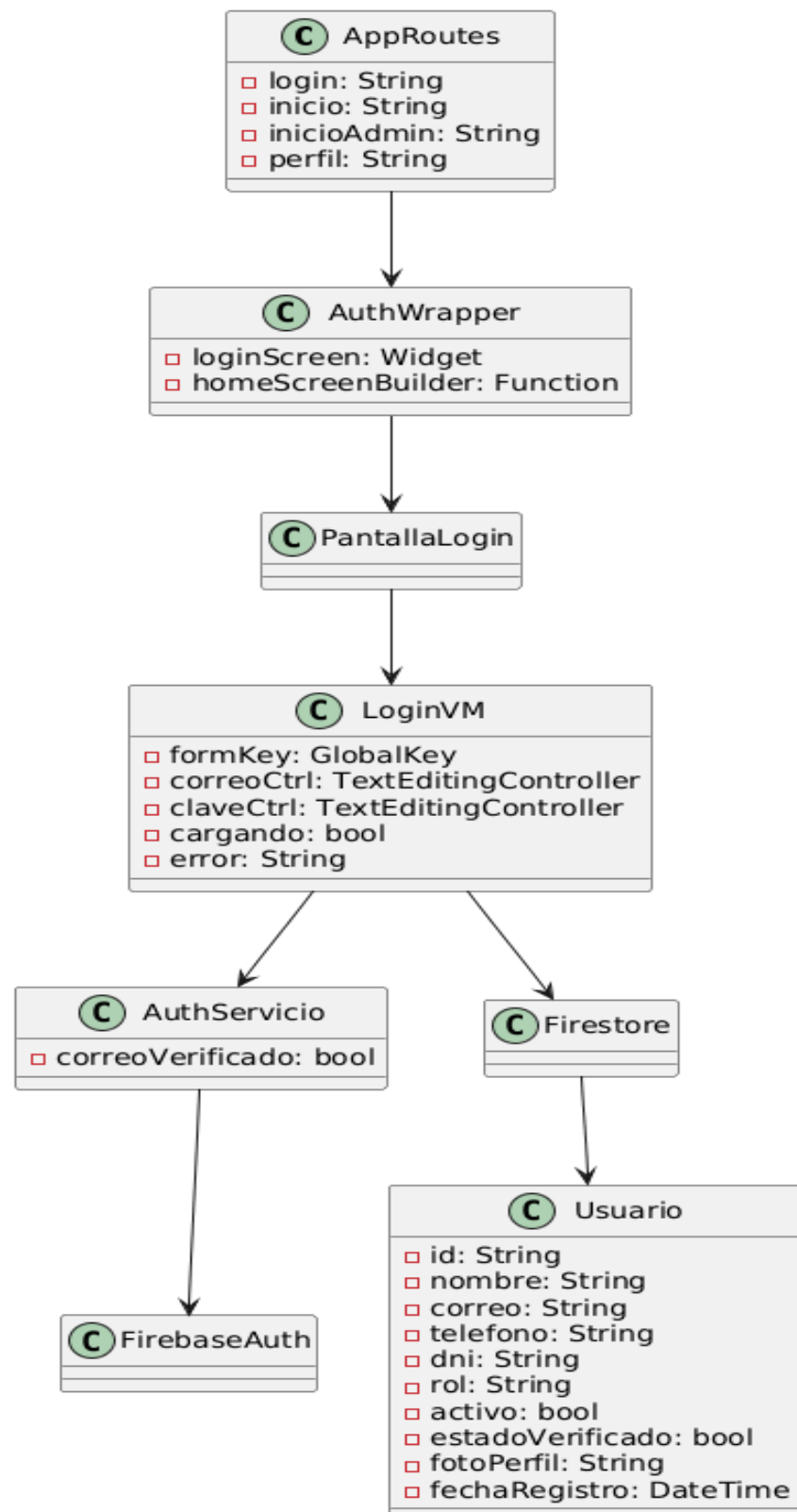
Figura 39: Diagrama de Clases de Autenticar Usuario fase análisis

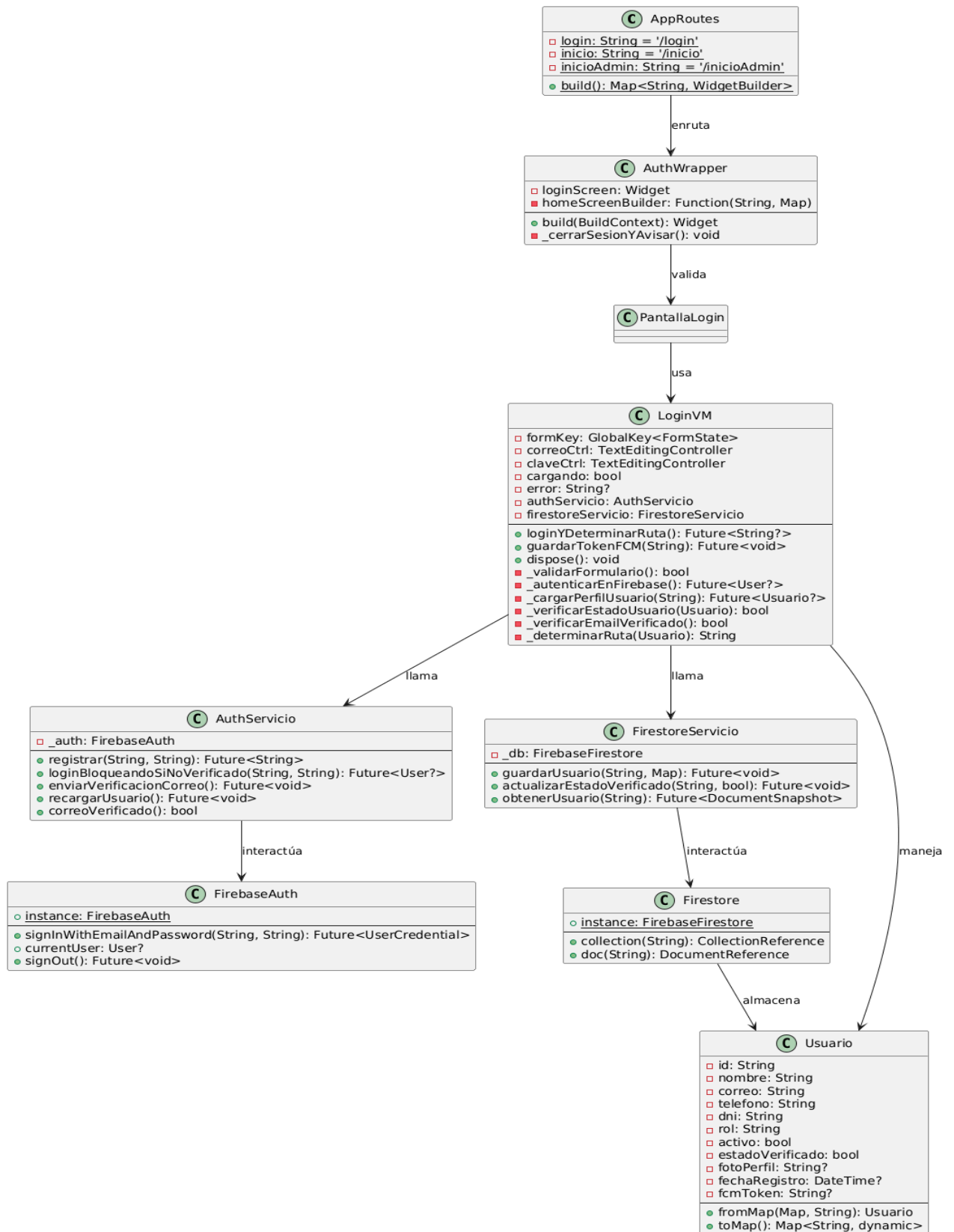
Figura 41: Diagrama de Clases de Autenticar Usuario fase diseño


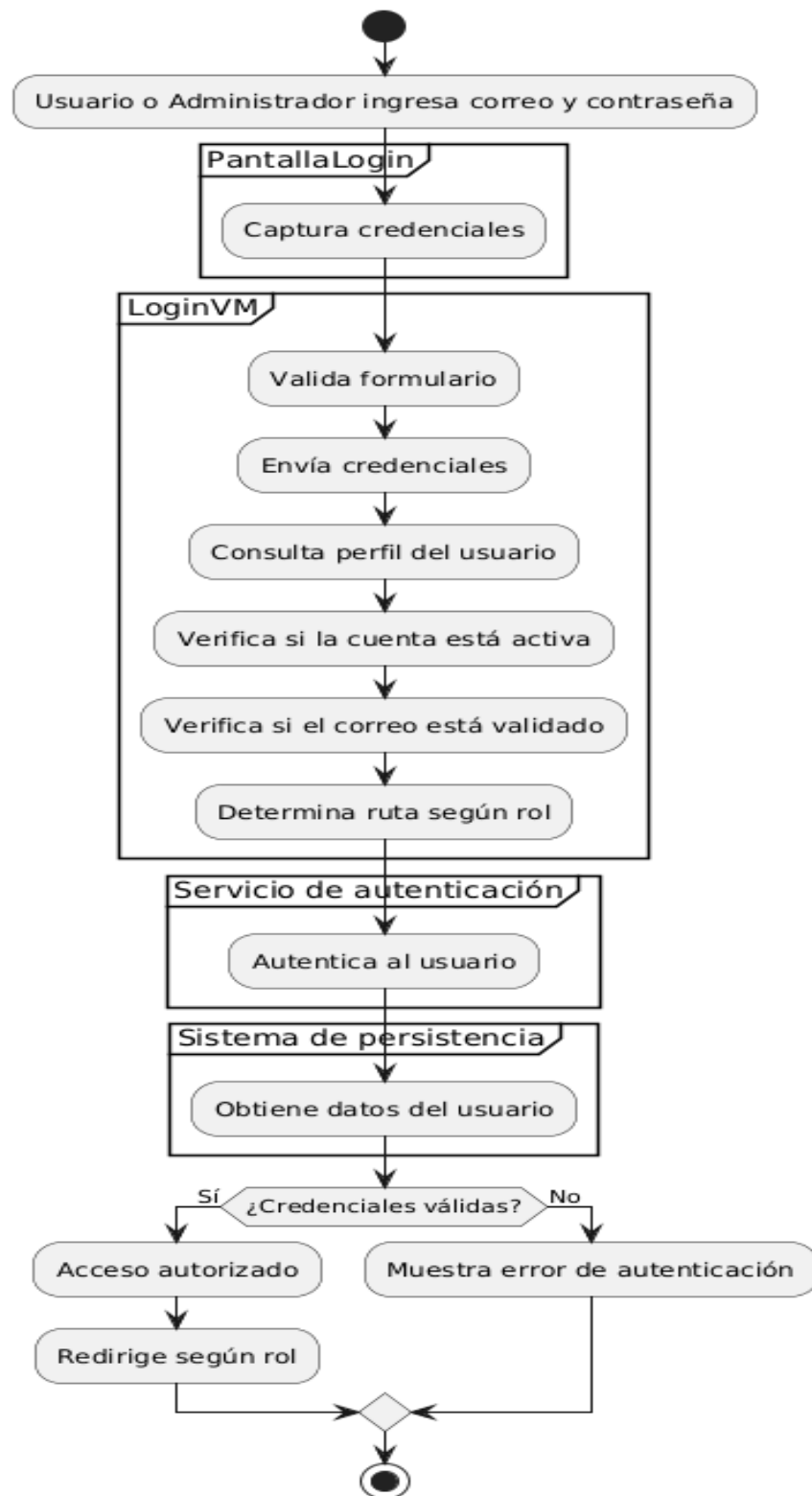
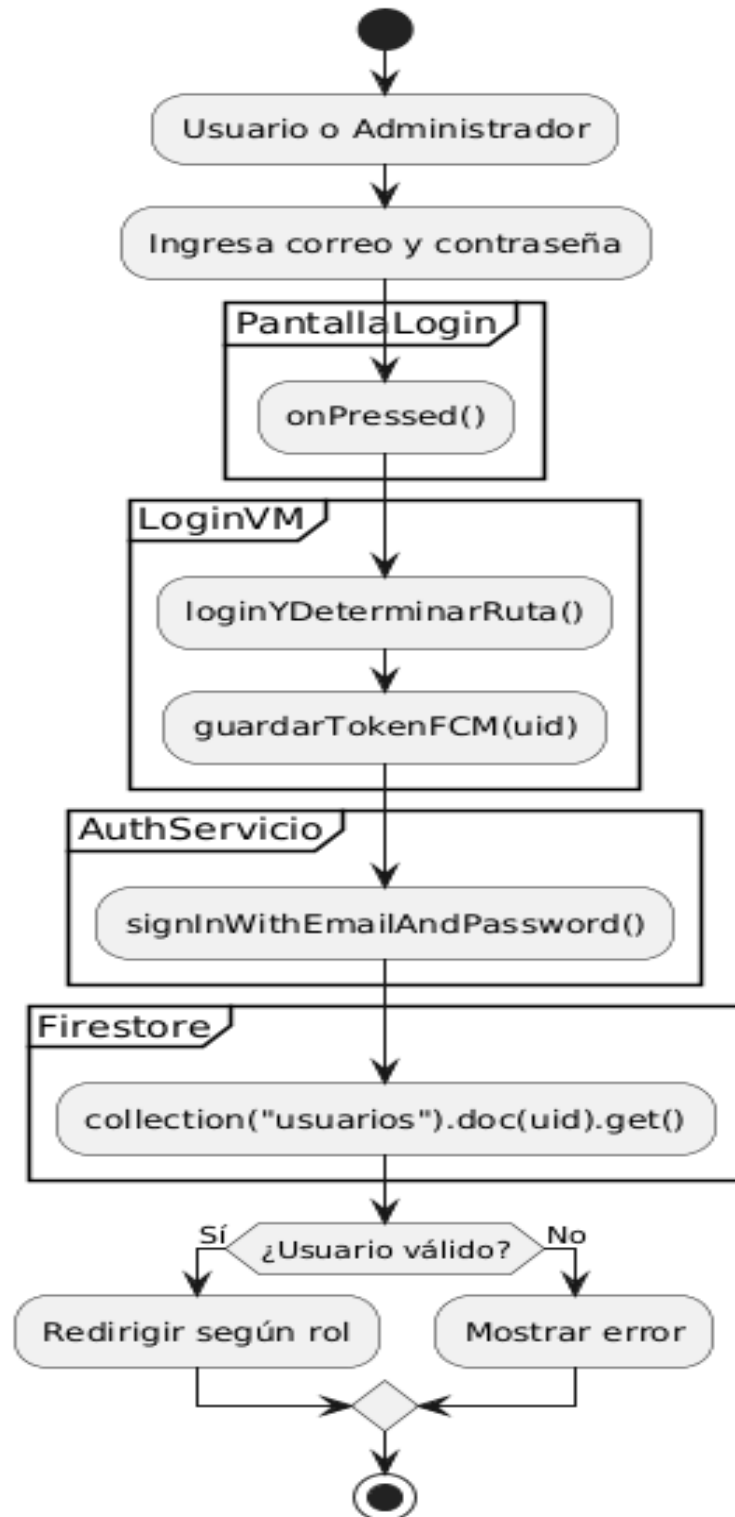
Figura 07: Diagrama de Actividades con objetos de Autenticar Usuario fase análisis

Figura : Diagrama de Actividades con objetos de Autenticar Usuario fase diseño



"Objetos de programación creados:

Front end:

pantalla_login.dart (UI y validación visual del inicio de sesión)

pantalla_inicio.dart (dashboard del usuario con tarjetas y menú)

pantalla_inicio_admin.dart (panel de administración)

Backend:

Backend - Vista Modelo (ViewModel):

login_vm.dart (lib/vistamodelo/auth/) - Validación de credenciales, consulta de perfil y redirección según rol

Backend - Servicios:

auth_servicio.dart (lib/servicios/) - Gestión de autenticación (Firebase)

firestore_servicio.dart (lib/servicios/) - Operaciones CRUD de usuario en base de datos.

app.dart (lib/) - Rutas y configuración general de la aplicación

auth_wrapper.dart (lib/utills/) - Control de sesión activa y bloqueo de acceso no autorizado

BD & Modelos:

usuario.dart (lib/modelo/) - Modelo del usuario: nombre, correo, teléfono, dni, rol, activo, fotoPerfil, fechaRegistro, estadoVerificado

Firestore Authentication - Autenticación de usuario : auth_servicio.dart, login_vm.dart .

Firestore Firestore - Base de datos de usuarios: firestore_servicio.dart, login_vm.dart.

Firestore Authentication: para login (email/password)

Modelo:**usuario.dart**

```
usuario.dart X
lib > modelo > usuario.dart > Usuario
1  import 'package:cloud_firestore/cloud_firestore.dart';
2
3  class Usuario {
4      final String id;
5      final String nombre;
6      final String correo;
7      final String telefono;
8      final String dni;
9      final String rol;
10     final bool estadoVerificado;
11     final String estadoRol;
12     final String? fotoPerfil;
13     final DateTime? fechaRegistro;
14     final String? fcmToken;
15     final bool activo;
16
17     Usuario({
18         required this.id,
19         required this.nombre,
20         required this.correo,
21         required this.telefono,
22         required this.dni,
23         required this.rol,
24         required this.estadoVerificado,
25         required this.estadoRol,
26         this.fotoPerfil,
27         this.fechaRegistro,
28         this.fcmToken
```

Vistas:**pantalla_inicio_admin.dart**

```
pantalla_inicio_admin.dart X
lib > vista > admin > pantalla_inicio_admin.dart > PantallaInicioAdmin
1  import 'package:flutter/material.dart';
2  import 'package:firebase_auth/firebase_auth.dart';
3  import 'package:cloud_firestore/cloud_firestore.dart';
4  import '../servicios/firebase_availability.dart' as fb_avail;
5
6  class PantallaInicioAdmin extends StatefulWidget {
7    const PantallaInicioAdmin({super.key});
8
9    @override
10   State<PantallaInicioAdmin> createState() => _PantallaInicioAdminState();
11 }
12
13 class _PantallaInicioAdminState extends State<PantallaInicioAdmin> {
14   bool _modoOscuro = false;
15   String _primerNombre = "Admin";
16   String? _fotoPerfil;
17   bool _datosCargados = false;
18
19   @override
20   void initState() {
21     super.initState();
22     _cargarDatosAdmin();
23   }
24
25   Future<void> _cargarDatosAdmin() async {
26     if (!fb_avail.firebaseAvailable) {
27       if (mounted) setState(() => _datosCargados = true);
```

pantalla_login.dart

```
🔍 pantalla_login.dart X

lib > vista > auth > 🔍 pantalla_login.dart > ...

1  import 'package:flutter/material.dart';
2  import 'package:flutter/services.dart';
3  import 'package:provider/provider.dart';
4  import '../vistamodelo/auth/login_vm.dart';
5
6  class PantallaLogin extends StatefulWidget {
7    const PantallaLogin({super.key});
8
9    @override
10   State<PantallaLogin> createState() => _PantallaLoginState();
11 }
12
13 class _PantallaLoginState extends State<PantallaLogin> {
14   late final LoginVM _vm;
15
16   @override
17   void initState() {
18     super.initState();
19     _vm = LoginVM();
20   }
21
22   @override
23   void dispose() {
24     _vm.dispose();
25     super.dispose();
26   }
27 }
```

pantalla_inicio.dart

```
pantalla_inicio.dart X
lib > vista > usuario > pantalla_inicio.dart > _PantallaInicioState
1  import 'package:flutter/material.dart';
2  import 'package:cloud_firestore/cloud_firestore.dart';
3  import 'package:firebase_auth/firebase_auth.dart';
4  import '../servicios/firebase_availability.dart' as fb_avail;
5  // import 'package:provider/provider.dart';
6  // import 'package:sos_mascotas/vistamodelo/notificacion/notificacion_vm.dart';
7  // import 'package:sos_mascotas/vista/chat/pantalla_chats_activos.dart';
8
9
10 class PantallaInicio extends StatefulWidget {
11   const PantallaInicio({super.key});
12
13   @override
14   State<PantallaInicio> createState() => _PantallaInicioState();
15 }
16
17 class _PantallaInicioState extends State<PantallaInicio> {
18   int _currentIndex = 0;
19   bool _modoOscuro = false;
20
21   String _primerNombre = "Usuario";
22   String? _fotoPerfil;
23   bool _datosCargados = false;
24
25   @override
26   void initState() {
27     super.initState();
```

VistaModelo:

login_vm.dart

```
login_vm.dart X
lib > vistamodelo > auth > login_vm.dart > LoginVM
1 import 'package:cloud_firestore/cloud_firestore.dart';
2 import 'package:firebase_messaging/firebase_messaging.dart';
3 import 'package:flutter/material.dart';
4 import 'package:firebase_auth/firebase_auth.dart';
5 import '../modelo/usuario.dart';
6
7 class LoginVM extends ChangeNotifier {
8   final formKey = GlobalKey<FormState>();
9   final correoCtrl = TextEditingController();
10  final claveCtrl = TextEditingController();
11
12  bool cargando = false;
13  String? error;
14
15  @override
16  void dispose() {
17    correoCtrl.dispose();
18    claveCtrl.dispose();
19    super.dispose();
20  }
21
22  Future<void> guardarTokenFCM(String uid) async {
23    try {
24      final token = await FirebaseMessaging.instance
25        .getToken()
26        .timeout(const Duration(seconds: 5));
27
28      if (token != null) {
```

Resultado:**Login:**



SOS Mascota

Encuentra a tu mascota perdida

Iniciar Sesión

 Correo electrónico

 Contraseña

[¿Olvidaste tu contraseña?](#)

Entrar

[¿No tienes una cuenta? Registrarse](#)

 **Contacto de Soporte**

 chitchitedanilo@gmail.com 

Toca el correo para copiarlo

Usuario:



SOS Mascota

Encuentra a tu mascota perdida

Iniciar Sesión

Correo electrónico

 bc2021070015@virtual.upt.pe

Contraseña



[¿Olvidaste tu contraseña?](#)

 Entrar

[¿No tienes una cuenta? Registrarse](#)

 **Contacto de Soporte**

 chitchitedanilo@gmail.com 

Toca el correo para copiarlo

 **¡Hola, Brian!**
Ayudemos a encontrar mascotas

 Buscar mascotas perdidas...

Acciones Rápidas

 Reportar Mascota

 Registrar Avistamiento

 Mapa Interactivo

Menú Principal

-  **Ver Mascotas Reportadas**
Explora todos los reportes >
-  **Mis Reportes**
Gestiona tus publicaciones >
-  **Chats**
Conversaciones activas >

 Inicio

 Tema

 Reportar

 Mapa

 Perfil

```
http://127.0.0.1:19130/?url=https://api.firebaseio.com/
Starting application from main method in: org-dartlang-app:///web_entrypoint.dart.
❌ Firebase App Check desactivado para entorno de desarrollo.
🌐 Modo Web: notificaciones locales deshabilitadas.
🔑 Intentando autenticación con Firebase para: bc2021070015@virtual.upt.pe
✅ Autenticación exitosa. UID: NcbPqoAKKkNoEYgz7wW7pI9aUj03
✅ Usuario cargado: Brian Danilo (Rol: usuario)
➡ Usuario con foto, navegando a /inicio
[EXCEPTION CAUGHT BY IMAGE RESOURCE SERVICE]
```

Admin:



SOS Mascota

Encuentra a tu mascota perdida

Iniciar Sesión

Correo electrónico

 chitedanilo@gmail.com

Contraseña



[¿Olvidaste tu contraseña?](#)

Entrar

¿No tienes una cuenta? [Registrarse](#)

 **Contacto de Soporte**

 chitchitedanilo@gmail.com 

 ¡Bienvenido, Brayan!

Panel de Administración

 6
Usuarios

 8
Reportes

 5
Perdidos

 3
Encontrados

Gestión Administrativa

 **Gestionar Usuarios**
Ver y administrar usuarios registrados

 **Gestionar Reportes**
Ver y moderar reportes de mascotas

 **Avistamientos**
Revisar avistamientos registrados

 **Estadísticas**
Ver análisis y reportes

```

scale: 1.0, webImplementationStrategy: never, headers: null)
✖ Error al guardar token FCM (ignorado): TimeoutException after 0:00:05.000000: Future not completed
🔒 Intentando autenticación con Firebase para: chitedanilo@gmail.com
✅ Autenticación exitosa. UID: Wg77LsCJnzbZK5Go94bipFuZtQt1
✅ Usuario cargado: Brayan Danilo (Rol: admin)
🔗 Navegando a /inicioAdmin
✖ Error al guardar token FCM (ignorado): TimeoutException after 0:00:05.000000: Future not completed

```

Caso de Uso 02: "Registrar reporte de mascota"

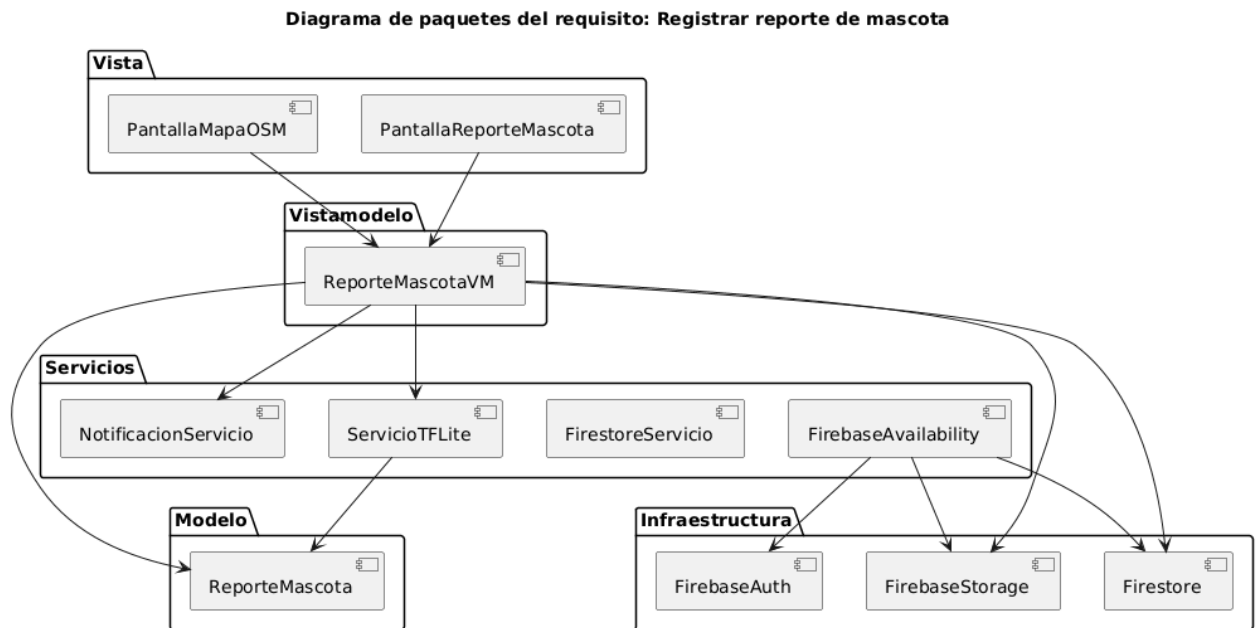


Figura 06: Diagrama de Caso de USO de Registrar reporte de mascota fase análisis

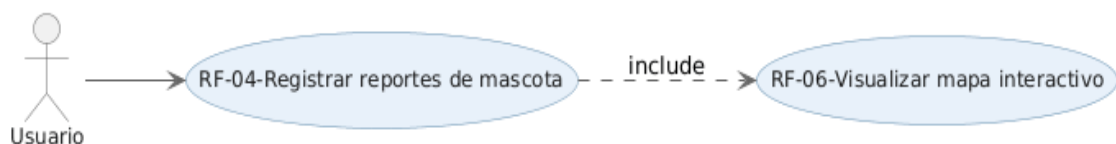


Figura 25: Diagrama de Caso de USO de Registrar reporte de mascota fase diseño

Figura 25: Diagrama de Caso de USO de Registrar reporte de mascota fase diseño

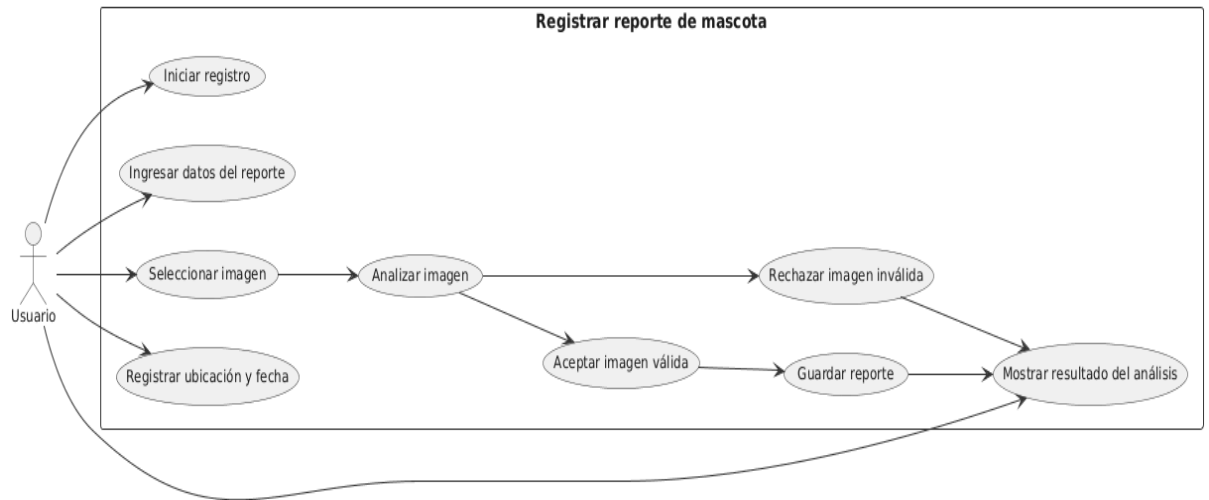


Figura 21: Diagrama de Caso de Secuencia de Registrar reporte de mascota fase análisis

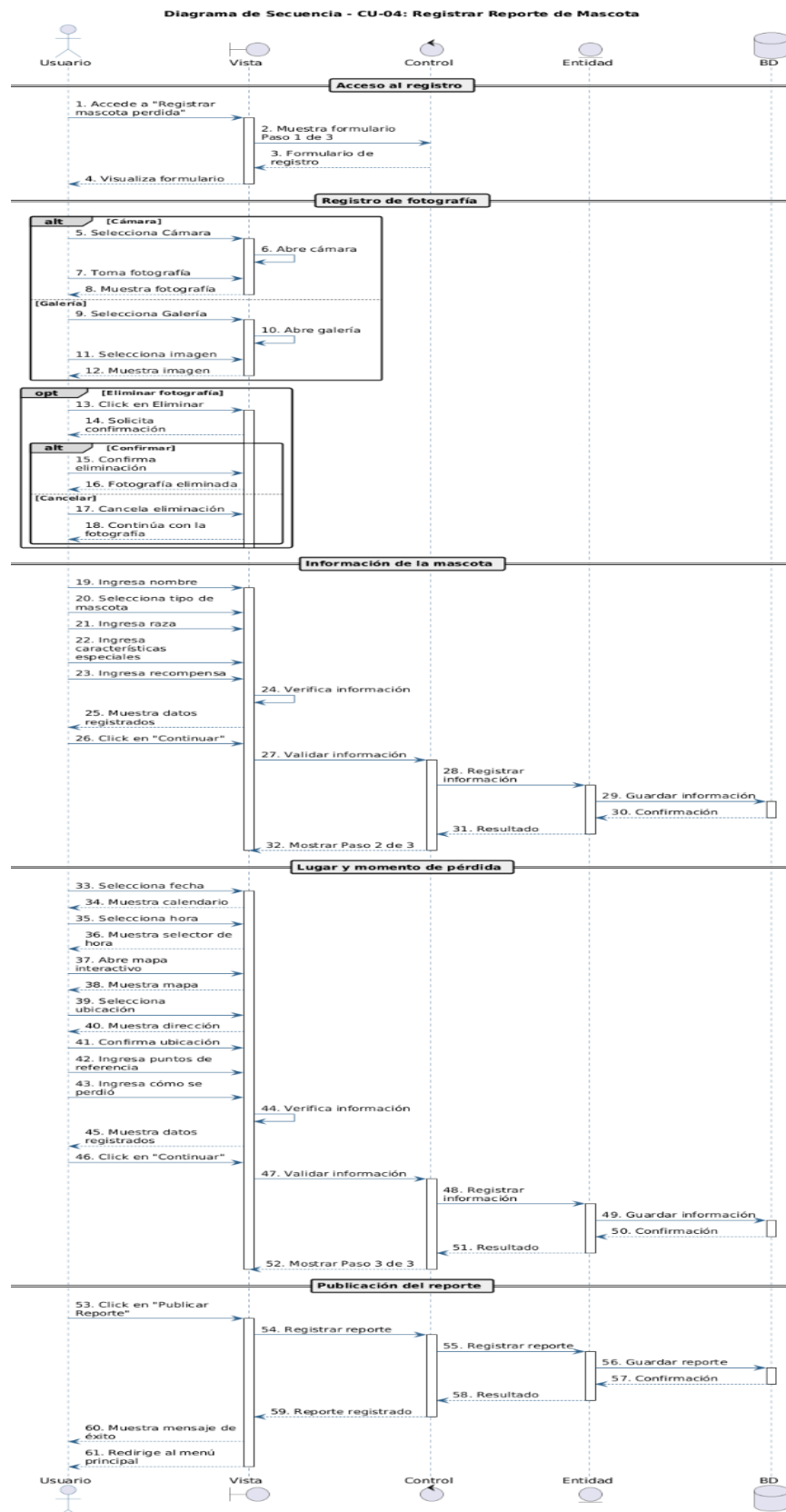


Figura 06: Diagrama de Caso de Secuencia de Registrar reporte de mascota fase diseño

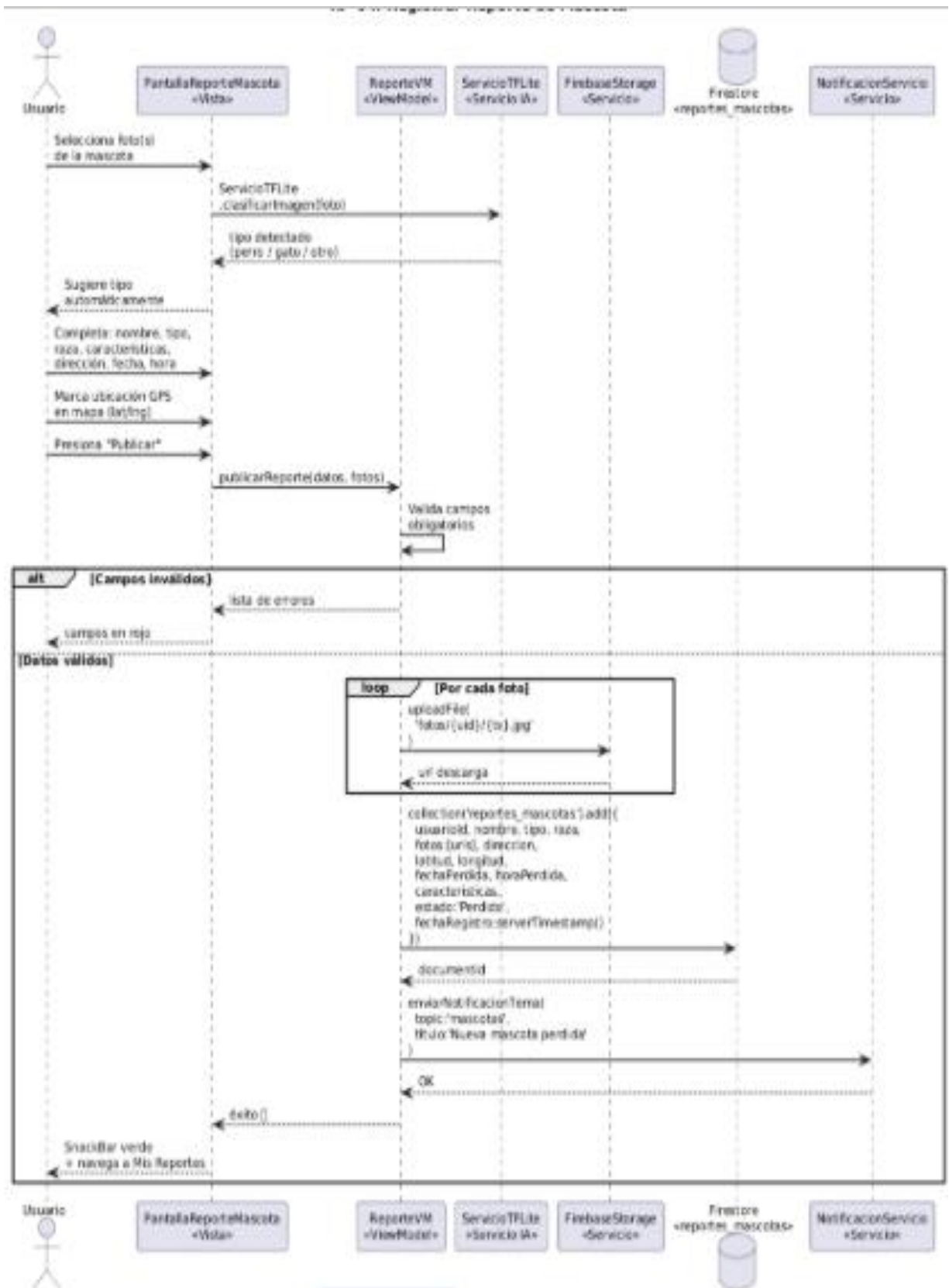


Figura 39: Diagrama de Caso de Clases de Registrar reporte de mascota fase análisis

Figura 39: Diagrama de Caso de Clases de Registrar reporte de mascota fase análisis

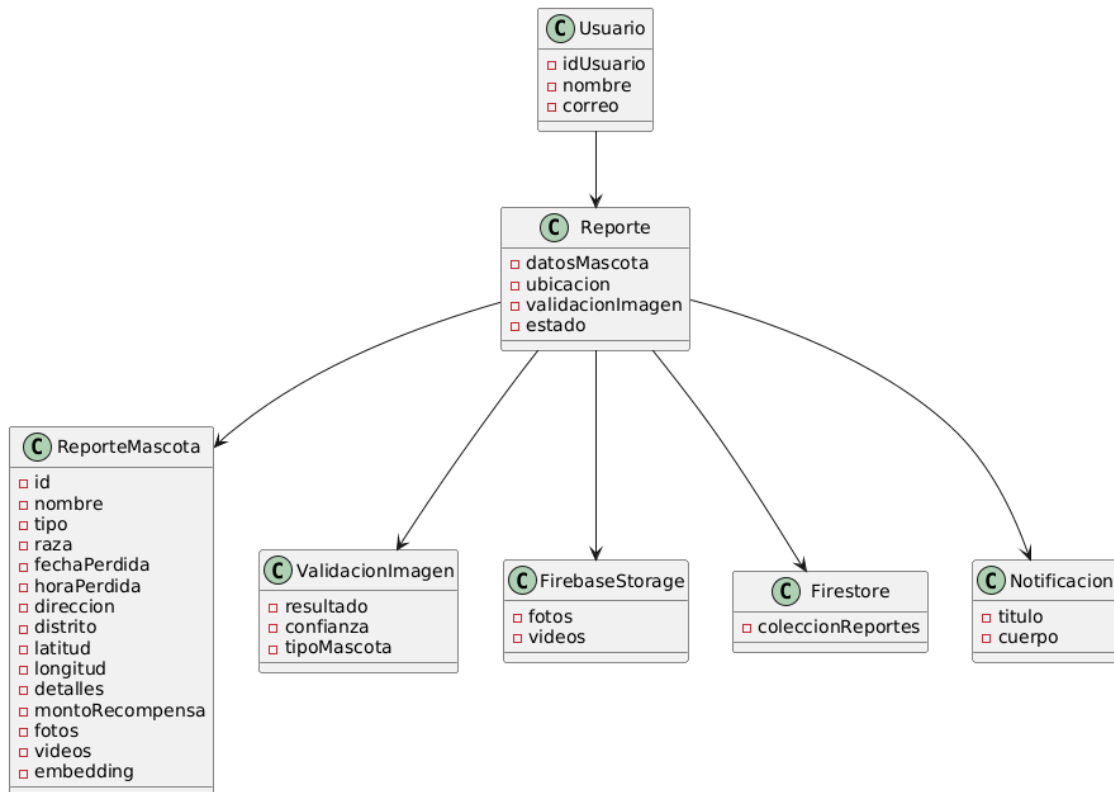


Figura 41: Diagrama de Caso de Clases de Registrar reporte de mascota fase diseño

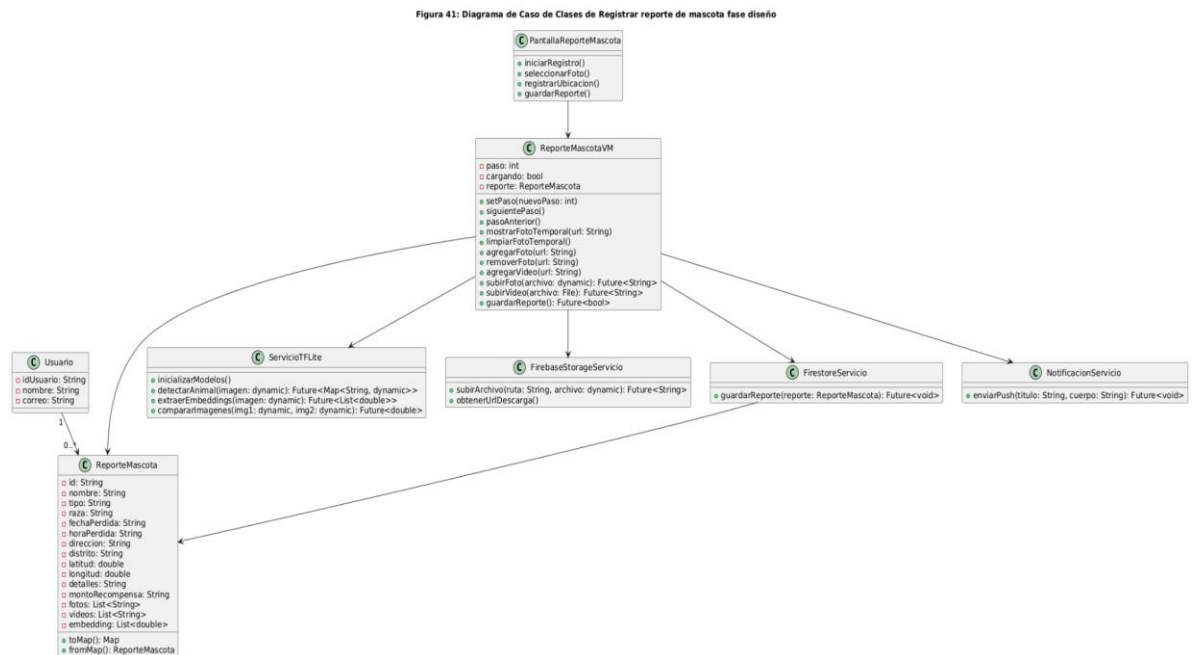
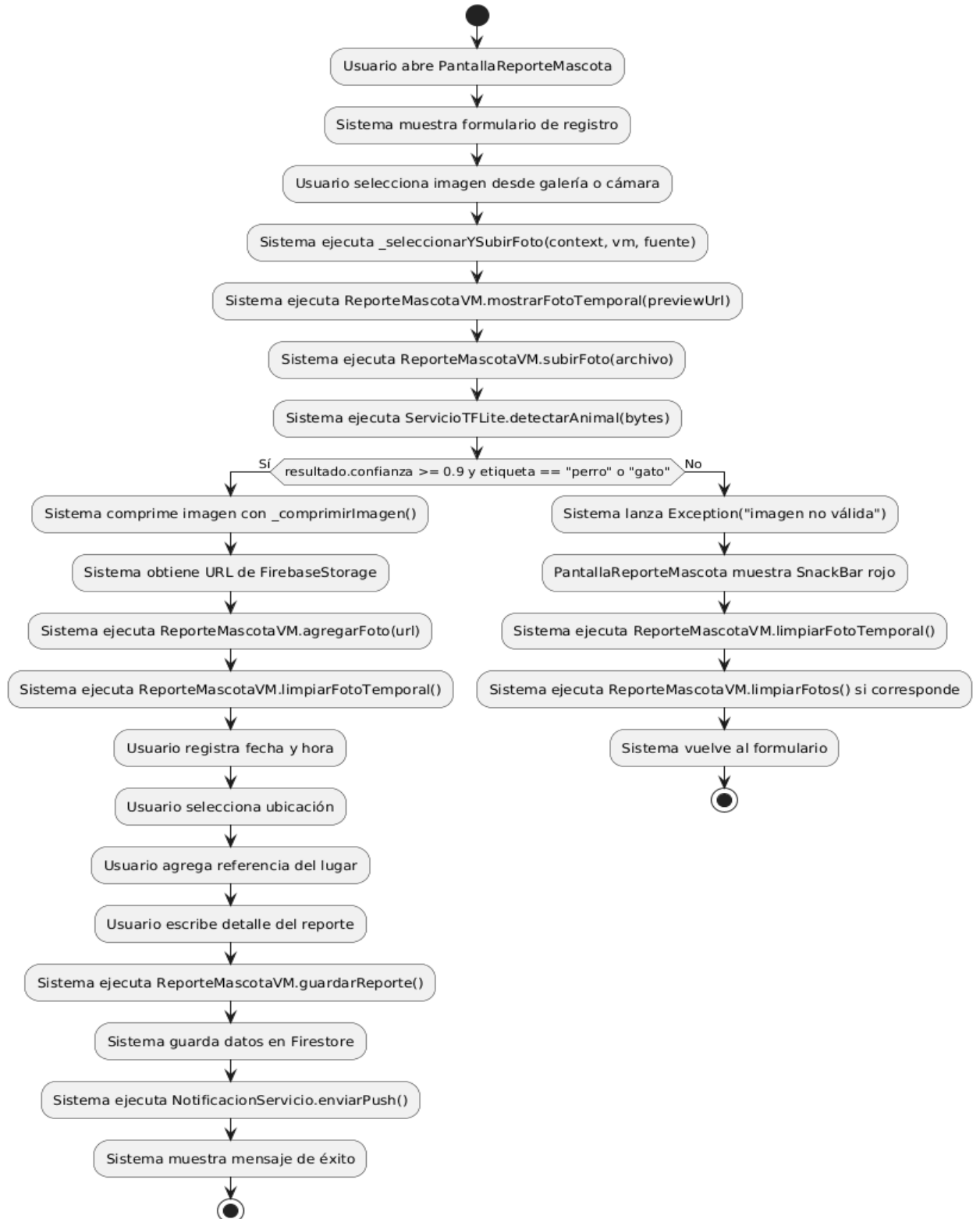


Figura 10: Diagrama de Actividades con Objetos de Registrar reporte de mascota fase análisis

Figura 10: Diagrama de Actividades con Objetos de Registrar reporte de mascota fase análisis**Figura 43: Diagrama de Actividades con Objetos de Registrar reporte de mascota fase diseño**

Figura 10: Diagrama de Actividades con Objetos de Registrar reporte de mascota fase diseño

Objetos de programación creados:

**Front end:**

pantalla_reporte_mascota.dart : interfaz principal para registrar el reporte de mascota (UI del formulario y flujo del registro).

video_recorte_page.dart (gestión del video si se adjunta)

Backend:

reporte_vm.dart (lógica del formulario, validación, carga de archivos y guardado)

servicio_tflite.dart (validación de la imagen con IA para detectar si es perro o gato)

notificacion_servicio.dart (envío de notificaciones push)

firestore_servicio.dart (persistencia en Firestore)

BD & Modelos:

reporte_mascota.dart (modelo del reporte con campos como nombre, tipo, raza, fechaPerdida, horaPerdida, direccion, distrito, fotos, videos, detalles, montoRecompensa).

Firebase Storage + Firestore**Firebase Storage:**

Se usa para guardar fotos y videos del reporte de mascota.

servicio_tflite.dart, reporte_vm.dart (FirebaseStorage.instance, .ref(), .child("reportes_mascotas"), putFile(...), getDownloadURL())

Firestore: Se usa para guardar la información del reporte: **reporte_mascota.dart, reporte_vm.dart**: FirebaseFirestore.instance, .collection("reportes_mascotas"), .doc(), .set(...)

Firestore colección reportes_mascotas: guardado del reporte junto con sus datos."

Resultado:



←

Reportar Mascota Perdida

Paso 1 de 3

Foto de tu mascota

Agregar foto de tu mascota

Toca para seleccionar desde galería o cámara

Cámara

Galería

Información de la mascota

Nombre de la mascota

Tipo de mascota

Raza

Analiza mi servicio tflite.

Cosa que no sea una mascota:

←

Paso 1 de 3

Foto de tu mascota

Agregar foto de tu mascota

Toca para seleccionar desde galería o cámara

Cámara

Galería

Información de la mascota

Nombre de la mascota

Tipo de mascota

Foto inválida: La imagen no es válida. Debe ser una foto clara de una mascota, como un perro o gato.

Una mascota:

Foto de tu mascota



✓ Imagen válida detectada y subida correctamente.

Reportar Mascota Perdida

Paso 1 de 3

←

Información de la mascota

Nombre de la mascota

Julian

Tipo de mascota

🐕 Perro

Raza

Pudgol

Características especiales

peludito chiquito

🏆 Recompensa de búsqueda (opcional)

Si deseas ofrecer una recompensa a quien encuentre tu mascota, completa los campos que desees.

💰 Monto de recompensa (ej. \$/. 200)

200

Atrás

Continuar



Reportar Mascota Perdida

Paso 2 de 3

Back

Lugar y momento de pérdida

Fecha de pérdida
29/08/2026

Hora aproximada
13:35

Seleccionar ubicación en el mapa

Abrir mapa interactivo

Dirección (puedes editar si lo deseas)
1410, Avenida Francisco Bolognesi, Urbanización Los Rosales, Vigil, Jesús María, Tacna, 23001, Perú

Puntos de referencia
Av.bolognesi

¿Cómo se perdió?
Se extravió en la calle

Reportar Mascota Perdida

Paso 3 de 3



Nombre: Julian

Tipo: Perro • Pudgol

Perdido: 29/08/2026 a las 13:35

Dirección: 1410, Avenida Francisco Bolognesi, Urbanización Los Rosales, Vigil, Jesús María, Tacna, 23001, Perú

Referencia: Av.bolognesi

Características especiales: peludito chiquito

Recompensa de búsqueda

Monto: S/. 200

Atrás

Publicar reporte

Buscar mascotas perdidas...

Acciones Rápidas

Reportar Mascota

Cerrar Sesión

Reporte guardado con éxito

Inicio Tema Reportar Mapa Perfil



```
WS
Starting application from main method in: org-dartlang-app:///web_entrypoint.dart.
🌐 Firebase no disponible en web; arrancando en modo offline.
✅ Modelos TFLite inicializados correctamente
🌐 Modo Web: notificaciones locales deshabilitadas.
🌐 Firebase no inicializado; omitiendo tareas de red.
TFLite: petLike=false tamaño=224x224
TFLite: rechazada como no mascota
VM: etiqueta=otro confianza=0.02
TFLite: petLike=false tamaño=224x224
TFLite: rechazada como no mascota
VM: etiqueta=otro confianza=0.02
TFLite: petLike=true tamaño=224x224
TFLite: aceptada como mascota
VM: etiqueta=perro confianza=0.99
[flutter_map]
```

Modelo:

reporte_mascota.dart

 reporte_mascota.dart Xlib > modelo >  reporte_mascota.dart > ...

```
1  class ReporteMascota {
2      String id;
3      String nombre;
4      String tipo;
5      String raza;
6      String características;
7      String fechaPerdida;
8      String horaPerdida;
9      String direccion;
10     String distrito; //  nuevo campo
11     double? latitud; //  nuevo campo
12     double? longitud; //  nuevo campo
13     String referencia;
14     String circunstancia;
15     String detalles;
16     // 🏆 Recompensa personal del dueño
17     String montoRecompensa; // ej. "S/. 200" o vacío si no ofrece
18     List<String> fotos; // imágenes
19     List<String> videos; // videos de máx 10 seg
20     List<double> embedding; //  nuevo campo IA
21
22     ReporteMascota({
23         this.id = "",
24         this.nombre = "",
25         this.tipo = "",
26         this.raza = "",
27         this.caracteristicas = "",
28         this.fechaPerdida = ""
```

servicio_tflite.dart

```
servicio_tflite.dart 1 X
lib > servicios > servicio_tflite.dart > ...
1 import 'dart:io';
2 import 'dart:typed_data';
3 import 'package:image/image.dart' as img;
4
5 class ServicioTFLite {
6   static bool modelosDisponibles = false;
7
8   static Future<void> inicializarModelos() async {
9     modelosDisponibles = true;
10  }
11
12  static Future<Map<String, dynamic>> detectarAnimal(dynamic imagen) async {
13    await inicializarModelos();
14
15    final bytes = _bytesDeImagen(imagen);
16    if (bytes == null || bytes.isEmpty) {
17      print('TFLite: imagen nula o vacía');
18      return {"etiqueta": "otro", "confianza": 0.0};
19    }
20
21    final decoded = img.decodeImage(bytes);
22    if (decoded == null) {
23      print('TFLite: no se pudo decodificar');
24      return {"etiqueta": "otro", "confianza": 0.0};
25    }
26
27    final foto = img.copyResize(decoded, width: 224, height: 224);
28    final netlike = _netlikeMascota(foto);
```

pantalla_reporte_mascota.dart

```
pantalla_reporte_mascota.dart X
lib > vista > reportes > pantalla_reporte_mascota.dart > _infoRow
1  import 'dart:convert';
2  import 'dart:io';
3  import 'package:flutter/foundation.dart';
4  import 'package:flutter/material.dart';
5  import 'package:image_picker/image_picker.dart';
6  import 'package:provider/provider.dart';
7  import 'package:sos_mascotas/vista/reportes/pantalla_mapa_osm.dart';
8  // ✓ Ruta corregida - el archivo está en lib/vistamodelo/ no en lib/vistamodelo/re
9  import 'package:sos_mascotas/vistamodelo/reporte_vm.dart';
10
11  /// Constantes visuales del wizard de reporte.
12  ///
13  /// Antes los colores, duraciones y tamaños estaban repetidos como
14  /// literales sueltos por todo el archivo. Centralizarlos aquí evita
15  /// "números mágicos" y hace que un cambio de paleta o de tiempos
16  /// se haga en un solo lugar.
17  class _ReporteEstilo {
18    _ReporteEstilo.();
19
20    // Colores
21    static const Color colorPrimario = Color(0xFF6366F1);
22    static const Color colorSecundario = Color(0xFF2563EB);
23    static const Color colorMapa = Color(0xFF4D9EF6);
24    static const Color colorRecompensa = Color(0xFFFFB300);
25    static const Color colorRecompensaTexto = Color(0xFFB45309);
26    static const Color colorGaleria = Color(0xFF7C3AED);
27    static const Color fondoCamara = Color(0xFFEFF6FF);
28    static const Color fondoGaleria = Color(0xFFE5E3E5);
```

reporte_vm.dart

```
reporte_vm.dart X
lib > vistamodelo > reporte_vm.dart > ReporteMascotaVM
1 import 'dart:convert';
2 import 'dart:io';
3 import 'dart:typed_data';
4 import 'package:flutter/material.dart';
5 import 'package:firebase_core/firebase_core.dart';
6 import 'package:cloud_firestore/cloud_firestore.dart';
7 import 'package:firebase_auth/firebase_auth.dart';
8 import 'package:firebase_storage/firebase_storage.dart';
9 import 'package:flutter_image_compress/flutter_image_compress.dart';
10 import 'package:path_provider/path_provider.dart';
11 import 'package:sos_mascotas/servicios/firebase_availability.dart' as fb_avail;
12 import 'package:sos_mascotas/servicios/notificacion_servicio.dart';
13 import 'package:sos_mascotas/servicios/servicio_tflite.dart';
14 import '../modelo/reporte_mascota.dart';
15
16 class ReporteMascotaVM extends ChangeNotifier {
17   int _paso = 0;
18   ReporteMascota reporte = ReporteMascota();
19   bool _cargando = false;
20   bool _disposed = false; // nuevo flag de control
21   String? _fotoTemporal;
22
23   // FormKeys para validaciones
24   final formKeyPaso1 = GlobalKey<FormState>();
25   final formKeyPaso2 = GlobalKey<FormState>();
26   final formKeyPaso3 = GlobalKey<FormState>();
27
28   // Getters
```

1.1. Análisis de calidad de código — Módulo Gestionar Comentarios

Esta sección corresponde a los archivos que forman el proyecto `demo_comentarios`, encargado del requerimiento CU01.

1.1.1. Archivo `main.dart`

El `main.dart` es el punto de partida de toda aplicación Flutter: es lo primero que se ejecuta. En el proyecto original, antes de llegar a cualquier pantalla, el usuario tenía que pasar por el login. Como esta demo debía abrir directamente la pantalla de comentarios, tuve que quitar esa dependencia sin romper la inicialización de Firebase ni las notificaciones.

Antes (en `SosMascota/lib/main.dart`):

```
import 'package:sos_mascotas/servicios/servicio_tflite.dart';

import 'firebase_options.dart';

import 'app.dart';
```

```
Future<void> _runNetworkDependentTasks() async {

  final user = FirebaseAuth.instance.currentUser;

  if (user != null) {

    // ... suscribe al usuario logueado al tema "mascotas"

    // ... guarda su token de notificaciones en Firestore

  }

}
```

```
Future<void> main() async {

  WidgetsFlutterBinding.ensureInitialized();
```

```
await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);

// ... inicializa TFLite, notificaciones, etc.

runApp(const MyApp());

_runNetworkDependentTasks();

}
```

Después (en demo comentarios/lib/main.dart):

```
// CAMBIO: import original era parte de la app completa con login.

// En esta demo se entra directamente a la pantalla de comentarios:

import 'vista/pantalla_comentarios.dart';

import 'firebase_options.dart';

Future<void> main() async {

  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);

  // ... configuración de notificaciones locales (igual que el original)

  runApp(const DemoComentariosApp());

}
```

Lo primero que hice fue eliminar por completo la función `_runNetworkDependentTasks()`. En el proyecto original esta función solo tenía sentido después de que el usuario iniciaba sesión, porque revisaba si había un usuario autenticado y, si lo había, lo suscribía a un tema de notificaciones y guardaba su token en Firestore. Como en esta demo no existe pantalla de login, esa función ya no se iba a ejecutar nunca con un propósito real, así que dejarla en el código habría sido una función muerta, es decir, código que ocupa espacio y genera confusión pero que nunca aporta nada. Por eso la quité en vez de dejarla ahí “por si acaso”.

También cambié el import de app.dart (que cargaba toda la aplicación con su sistema de rutas y login) por un import directo a vista/pantalla_comentarios.dart, que es la única pantalla que esta demo necesita mostrar. Dejé un comentario justo arriba de ese import explicando el motivo del cambio, tal como pide la buena práctica de comentar el porqué y no solo el qué.

1.1.2. Archivo vista/pantalla_comentarios.dart

Esta es la pantalla principal del módulo, con 835 líneas de código que manejan la lista de comentarios, el formulario para escribir uno nuevo, los likes y las respuestas. Decidí no tocar la lógica interna porque ya funcionaba correctamente y cualquier cambio ahí podía introducir errores nuevos; el único ajuste necesario fue en los imports.

Antes:

```
import '../servicios/notificacion_servicio.dart';
```

```
import '../utils/fecha_utils.dart';
```

Después:

```
//          CAMBIO:          import          original          era  
'package:sos_mascotas/servicios/notificacion_servicio.dart'
```

```
// Se actualiza al nuevo nombre de paquete demo_comentarios:
```

```
import '../servicios/notificacion_servicio.dart';
```

```
// CAMBIO: import original era '../utils/fecha_utils.dart'
```

```
// Se actualiza a la ruta relativa del nuevo proyecto:
```

```
import '../utils/fecha_utils.dart';
```

El motivo de este cambio es sencillo: en el proyecto original, la carpeta vista/ estaba dentro de una subcarpeta reportes/, entonces para llegar a servicios/ había que subir dos niveles (../..). En el proyecto nuevo la estructura de carpetas es más plana, así que solo hace falta subir un nivel (../). Si hubiera dejado la ruta vieja, el proyecto simplemente no habría compilado. Dejé el comentario explicando de dónde venía el import original para que quede clara la trazabilidad del cambio.

1.1.3. Archivo vistamodelo/comentarios_viewmodel.dart

Este archivo es el ViewModel: se encarga de pedirle a Firestore el stream de comentarios ordenados por fecha. Aquí también el único cambio fue de rutas de import, porque la lógica (el método obtenerComentarios()) seguía siendo válida tal cual.

Antes:

```
import '../modelo/comentario_model.dart';
```

Después:

```
// CAMBIO: import original era 'package:sos_mascotas/modelo/comentario_model.dart'
```

```
// Se actualiza al nuevo nombre de paquete demo_comentarios:
```

```
import '../modelo/comentario_model.dart';
```

1.1.4. Archivo servicios/notificacion_servicio.dart

Este archivo es el que más cambió a nivel de diseño, y por eso me detengo más en explicarlo. Su trabajo es enviar notificaciones push con Firebase Cloud Messaging y guardar un registro de esas notificaciones en Firestore.

Antes:

```
import 'package:sos_mascotas/modelo/notificacion.dart';
```

```
class NotificacionServicio {
```

```
  static const _scopes = ['https://www.googleapis.com/auth/firebase.messaging'];
```

```
  static const _jsonKeyPath = 'assets/keys/service_account.json';
```

```
  static final _db = FirebaseFirestore.instance;
```

```
  static Future<void> guardarNotificacion(Notificacion notif) async {
```

```
    await _db.collection('notificaciones').add(notif.toMap());
```

```
}

static Stream<List<Notificacion>> obtenerNotificaciones(String usuarioId) {
return _db.collection('notificaciones')

.where('usuarioId', isEqualTo: usuarioId)

.orderBy('fecha', descending: true)

.snapshots()

.map((snapshot) => snapshot.docs

.map((doc) => Notificacion.fromMap(doc.id, doc.data()))

.toList());

}

}
```

Después:

```
/// Servicio para enviar y gestionar notificaciones push (FCM) y en Firestore.

///

/// Archivo copiado íntegramente de:

/// [SosMascota/lib/servicios/notificacion_servicio.dart]

class NotificacionServicio {

// Scopes requeridos por la API de FCM v1

static const _scopes = ['https://www.googleapis.com/auth/firebase.messaging'];

static const _jsonKeyPath = 'assets/keys/service_account.json';

static final _db = FirebaseFirestore.instance;
```

```
/// Guarda una notificación en la colección 'notificaciones' de Firestore.

static Future<void> guardarNotificacion(Map<String, dynamic> notif) async {

await _db.collection('notificaciones').add(notif);

}

/// Obtiene las notificaciones de un usuario en tiempo real.

static Stream<List<Map<String, dynamic>>> obtenerNotificaciones(

String usuarioId,

) {

return _db.collection('notificaciones')

.where('usuarioId', isEqualTo: usuarioId)

.orderBy('fecha', descending: true)

.snapshots()

.map((snapshot) => snapshot.docs.map((doc) => doc.data()).toList());

}

}
```

En el original, este servicio dependía de una clase llamada Notificacion (definida en modelo/notificacion.dart) que tenía sus propios métodos toMap() y fromMap(). El problema es que ese modelo no formaba parte de los archivos que necesitaba esta demo, así que si lo dejaba tal cual, hubiera tenido que copiar también esa clase y todo lo que ella usara, inflando el proyecto con código que no es parte del requerimiento CU01.

Por eso decidí “desacoplar” el servicio de esa clase específica y hacer que trabaje directamente con Map<String, dynamic>, que es justamente el formato nativo que usa Firestore para guardar y leer documentos. Esto no es una simplificación que pierda funcionalidad: el servicio guarda y lee exactamente los mismos datos que antes, solo que ya no obliga a tener una

clase extra para poder compilar. Además, agregué un comentario de documentación (con /// en vez de //) en la parte de arriba de la clase, explicando de dónde viene el archivo original, y comentarios cortos en cada método explicando qué hace, algo que en la versión original no existía.

2.1. Análisis de calidad de código — Módulo Registrar Avistamiento

Esta sección corresponde a los archivos del proyecto demo_avistamiento, para el requerimiento CU02. Este módulo fue el que más problemas de calidad tenía en su versión original, principalmente por el manejo de memoria.

2.1.1. Archivo main.dart

Igual que en el módulo de comentarios, tuve que cambiar el punto de entrada para que la aplicación abra directamente el formulario de avistamiento, además de asegurarme de que los modelos de inteligencia artificial (TensorFlow Lite) se cargaran correctamente desde el inicio.

Después (demo_avistamiento/lib/main.dart):

```
// CAMBIO: import original inicializaba TFLite desde main.dart con un import

// 'package:sos_mascotas/servicios/servicio_tflite.dart'

// Se actualiza al nuevo nombre de paquete demo_avistamiento:

import 'servicios/servicio_tflite.dart';

import 'vista/pantalla_avistamiento.dart';

Future<void> main() async {

  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);

  // 2. Inicializa modelos TFLite (operación LOCAL sobre assets)
```

```
// Es crítico que no bloquee la app si falla la carga del archivo.  
  
try {  
  
  await ServicioTFLite.inicializarModelos();  
  
  debugPrint("Modelos TFLite inicializados correctamente");  
  
} catch (e) {  
  
  debugPrint("Error al inicializar modelos TFLite: $e");  
  
}  
  
// ... notificaciones locales igual que el original  
  
runApp(const DemoAvistamientoApp());  
  
}
```

Aquí mantuve algo que sí estaba bien hecho en el original y que quise resaltar: la carga de los modelos TFLite está dentro de un bloque try/catch. Eso es una buena práctica porque, si por alguna razón el archivo del modelo de IA no se puede leer, la aplicación no se cierra de golpe, sino que registra el error con debugPrint y sigue funcionando (aunque después la validación de fotos falle). Dejé un comentario aclarando por qué es importante que ese bloque no detenga la app.

2.2.2. Archivo vista/pantalla_avistamiento.dart — el cambio más importante

Este fue el archivo donde encontré el problema de calidad más serio de todo el proyecto: una fuga de memoria por controladores de texto que nunca se liberaban.

Antes:

```
class _FormularioAvistamientoState extends State<_FormularioAvistamiento> {  
  
  final formKey = GlobalKey<FormState>();  
  
  final picker = ImagePicker();  
  
}
```



```
late TextEditingController direccionCtrl;  
  
late TextEditingController fechaCtrl;  
  
late TextEditingController horaCtrl;  
  
late TextEditingController descripcionCtrl;
```

```
File? fotoSeleccionada;
```

```
@override  
  
void initState() {  
    super.initState();  
  
    direccionCtrl = TextEditingController();  
    fechaCtrl = TextEditingController();  
    horaCtrl = TextEditingController();  
    descripcionCtrl = TextEditingController();  
}  
  
// No existía el método dispose(): los 4 controladores  
// quedaban abiertos en memoria aunque la pantalla se cerrara.  
}
```

Después:

```
class _FormularioAvistamientoState extends State<_FormularioAvistamiento> {  
  
    final formKey = GlobalKey<FormState>();  
  
    final picker = ImagePicker();  
  
  
    late TextEditingController direccionCtrl;  
  
    late TextEditingController fechaCtrl;  
  
    late TextEditingController horaCtrl;  
  
    late TextEditingController descripcionCtrl;  
  
  
    File? fotoSeleccionada;  
  
  
    @override  
  
    void initState() {  
  
        super.initState();  
  
        direccionCtrl = TextEditingController();  
  
        fechaCtrl = TextEditingController();  
  
        horaCtrl = TextEditingController();  
  
        descripcionCtrl = TextEditingController();  
  
    }  
  
    // CAMBIO / MEJORA: Liberación adecuada de recursos para  
  
    // prevenir fugas de memoria
```

```
@override  
  
void dispose() {  
  
  direccionCtrl.dispose();  
  
  fechaCtrl.dispose();  
  
  horaCtrl.dispose();  
  
  descripcionCtrl.dispose();  
  
  super.dispose();  
  
}  
}
```

Para explicarlo de la forma más simple posible: un `TextEditingController` es el objeto que Flutter usa para “escuchar” lo que el usuario escribe en un campo de texto (la dirección, la fecha, la hora y la descripción, en este caso). Cada vez que se crea uno de estos objetos con `TextEditingController()`, Flutter reserva un espacio en memoria para él. Ese espacio se debe devolver cuando la pantalla ya no se necesita, y la forma correcta de hacerlo es llamando al método `dispose()` de cada controlador dentro del método `dispose()` del propio `State`.

En el código original, el método `initState()` sí creaba los cuatro controladores, pero en ningún lugar del archivo existía un método `dispose()` que los cerrara. Esto significa que, cada vez que un usuario entraba y salía de la pantalla de avistamiento varias veces (por ejemplo, para probar el formulario, cancelar y volver a intentar), la aplicación iba acumulando controladores “fantasma” en la memoria del teléfono, sin ninguna forma de liberarlos hasta que la aplicación completa se cerrara. Con una pantalla esto casi no se nota, pero es exactamente el tipo de error que, en una aplicación más grande o usada durante mucho tiempo, termina generando que el celular se ponga lento o que la app se cierre sola por falta de memoria.

La corrección que apliqué fue agregar el método `dispose()`, llamando al método `dispose()` de cada uno de los cuatro controladores (`direccionCtrl`, `fechaCtrl`, `horaCtrl` y `descripcionCtrl`) y, al final, llamando también a `super.dispose()`, que es obligatorio para que Flutter termine de limpiar el resto del ciclo de vida del widget. Dejé además un comentario indicando que se trata de una mejora, no solo de un cambio de estructura, precisamente para dejar

constancia de por qué se agregó código que no existía en el original.

El segundo cambio de este archivo fue en la forma en que se abre el mapa para elegir la ubicación:

Antes:

```
final resultado = await Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (_) => const PantallaMapaOSM(),  
  ),  
);
```

Después:

```
final resultado = await Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (_) => const PantallaMapaOSM(  
      esAvistamiento: true,  
    ),  
  ),  
);
```

La pantalla del mapa (PantallaMapaOSM) la reutilizo tanto para cuando alguien reporta una mascota perdida como para cuando alguien registra un avistamiento, y la diferencia visual es el color del marcador en el mapa. En el original no se le pasaba ningún parámetro que indicara en qué modo se estaba usando el mapa, así que siempre se veía con el color por defecto. Al agregar `esAvistamiento: true` dejo explícito, con una variable booleana bien nombrada, en qué contexto se está usando ese componente reutilizable, lo cual es justamente una variable con __

propósito claro en vez de dejar que el componente “adivine” su comportamiento.

2.2.3. Archivo vistamodelo/avistamiento_vm.dart

Este archivo contiene toda la lógica de negocio del avistamiento: comprimir la imagen, validar con TFLite, subir a Storage, calcular la distancia con la fórmula de Haversine y comparar imágenes por similitud de coseno. Aquí, otra vez, la lógica no cambió en nada; solo corregí las rutas de import.

Antes:

```
import 'package:sos_mascotas/servicios/notificacion_servicio.dart';  
  
import 'package:sos_mascotas/servicios/servicio_tflite.dart';  
  
import '../modelo/avistamiento.dart';
```

Después:

```
// CAMBIO: import original era  
'package:sos_mascotas/servicios/notificacion_servicio.dart'  
  
// Se actualiza al nuevo nombre de paquete demo_avistamiento:  
  
import '../servicios/notificacion_servicio.dart';  
  
// CAMBIO: import original era 'package:sos_mascotas/servicios/servicio_tflite.dart'  
  
// Se actualiza al nuevo nombre de paquete demo_avistamiento:  
  
import '../servicios/servicio_tflite.dart';  
  
// CAMBIO: import original era '../modelo/avistamiento.dart'  
  
// Se actualiza a la ruta relativa del nuevo proyecto:  
  
import '../modelo/avistamiento.dart';
```

El motivo es el mismo que ya expliqué en los otros archivos: el proyecto original usaba el nombre de paquete `package:sos_mascotas/...`, que solo Dart reconoce cuando el archivo vive dentro de ese proyecto en particular (ese nombre viene definido en su `pubspec.yaml`). Como `demo_avistamiento` es ahora un proyecto propio con su propio `pubspec.yaml`, esas rutas absolutas

ya no existen y hay que reemplazarlas por rutas relativas (con ../) que apuntan a la nueva ubicación de cada archivo dentro de la misma carpeta lib/. Este tipo de cambio reduce el acoplamiento del módulo con el proyecto principal: ahora demo_avistamiento no necesita que exista ningún paquete llamado sos_mascotas para poder compilar.

3. Archivos que se mantuvieron sin cambios

No todos los archivos necesitaban corrección. Los siguientes se copiaron de forma exacta, sin modificar ni una sola línea, porque ya cumplían con los criterios de calidad revisados y no tenían ninguna dependencia de rutas que corregir:

Archivo	Origen en el proyecto principal	Observación
demo_comentarios/lib/firebase_options.dart	lib/firebase_options.dart	Mismas claves y bucket de Firebase
demo_comentarios/lib/utills/fecha_utils.dart	lib/utills/fecha_utils.dart	Formateo de fecha relativa idéntico
demo_comentarios/lib/modelo/comentario_model.dart	lib/modelo/comentario_model.dart	Mismos campos: likes, shares, etc.
demo_avistamiento/lib/firebase_options.dart	lib/firebase_options.dart	Mismas claves y bucket de Firebase
demo_avistamiento/lib/modelo/avistamiento.dart	lib/modelo/avistamiento.dart	Mismo modelo y serializadores
demo_avistamiento/lib/servicios/servicio_tflite.dart	lib/servicios/servicio_tflite.dart	Mismos pesos e inferencia TFLite
demo_avistamiento/lib/vista/pantalla_mapa_osm.dart	lib/vista/reportes/pantalla_mapa_osm.dart	Mismo mapa (FlutterMap + Nominatim Tacna)

Menciono esta tabla porque también forma parte de una revisión de calidad honesta: no cambié código solo por cambiar. Si un archivo ya estaba bien y no dependía de rutas del proyecto principal, lo dejé exactamente igual, evitando el riesgo de introducir un error nuevo en algo que ya funcionaba.

**4. Cuadro resumen de la revisión de calidad**

Criterio evaluado	¿Qué encontré en el original?	¿Qué corregí?
Comentarios explicativos	La mayoría de archivos no explicaba el motivo de sus decisiones de diseño.	Agregué comentarios de tipo <code>// CAMBIO</code> explicando el origen y motivo de cada modificación, y comentarios <code>///</code> de documentación en las clases.
Nombres de variables	El parámetro <code>esAvistamiento</code> no existía; el mapa no sabía en qué modo se usaba.	Agregué el parámetro booleano <code>esAvistamiento</code> : <code>true</code> con nombre claro y propósito explícito.
Código que ya no se usa	La función <code>_runNetworkDependentTasks()</code> dependía de un login que no existe en la demo.	Eliminé la función por completo en vez de dejarla como código muerto.
Gestión de memoria	Los 4 <code>TextEditingController</code> de <code>pantalla_avistamiento.dart</code> se creaban pero nunca se liberaban.	Agregué el método <code>dispose()</code> liberando los 4 controladores y llamando a <code>super.dispose()</code> .
Acoplamiento entre módulos	Varios archivos dependían del paquete <code>package:sos_mascotas/...</code> , exclusivo del proyecto principal.	Cambié a rutas relativas propias de cada proyecto, y en <code>notificacion_servicio.dart</code> eliminé la dependencia de la clase <code>Notificacion</code> usando <code>Map<String, dynamic></code> .
Funcionalidad	—	Verifiqué que las reglas de negocio (umbral <code>TFLite</code> 95 %, distancia Haversine 9 km, similitud de imagen 50 %, notificaciones FCM) siguieran funcionando exactamente igual.

5. Trazabilidad de requerimientos: de los diagramas al código

Además de revisar la calidad del código, en este curso también nos piden demostrar la trazabilidad, es decir, poder seguirle la pista a un requerimiento desde que se analiza (con diagramas UML de análisis), pasa por el diseño (con diagramas UML de diseño) y termina convertido en clases y archivos de código reales. Para ordenar esto elaboré una matriz de trazabilidad que conecta cada caso de uso con sus figuras correspondientes.

Una aclaración importante sobre cómo trabajé estos diagramas: como los dos casos de uso de este proyecto son relativamente sencillos y no tuvieron cambios estructurales grandes entre la fase de análisis y la fase de diseño, el contenido de cada diagrama de análisis y su correspondiente diagrama de diseño terminó siendo el mismo. Por eso, en la matriz cada diagrama aparece con dos números de figura (por ejemplo, “Figura 01/06”): el primero corresponde a la fase de análisis y el segundo a la fase de diseño. En las siguientes dos secciones muestro cada diagrama una sola vez, indicando ambos números de figura, para explicar con el mismo detalle su versión de análisis y su versión de diseño.

5.1. Caso de uso CU01 — Gestionar Comentarios

Los objetos de programación que resultaron de este caso de uso, ya en la fase de implementación, son los siguientes: en la capa de vista, el archivo pantalla_comentarios.dart con la clase PantallaComentariosReporte; en la capa de vistamodelo, comentarios_viewmodel.dart con la clase ComentariosViewModel; en la capa de modelo, comentario_model.dart con la clase Comentario; en servicios, notificacion_servicio.dart; y en utilidades, fecha_utils.dart. Estos archivos usan las colecciones de Firestore reportes_mascotas/{id}/comentarios y avistamientos/{id}/comentarios, además de usuarios y notificaciones, y la carpeta de Storage comentarios_reportes/{uid}-{timestamp}.

5.2. Caso de uso CU02 — Registrar Avistamiento

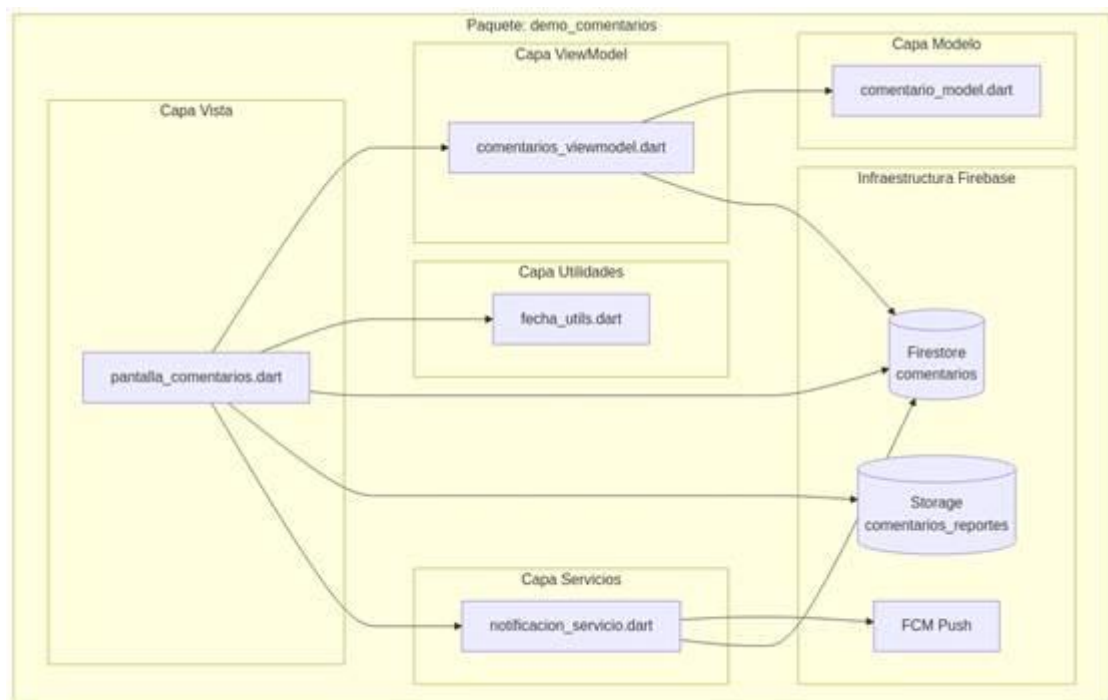
De manera similar, este caso de uso se tradujo en los archivos pantalla_avistamiento.dart y pantalla_mapa_osm.dart en la vista; avistamiento_vm.dart en el vistamodelo; avistamiento.dart en el modelo; y servicio_tflite.dart junto con notificacion_servicio.dart en la capa de servicios. Este módulo además necesita dos archivos adicionales que no son código sino modelos de inteligencia artificial: animales.tflite y _

extractor_animales.tfflite, guardados como assets del proyecto. Usa la colección avistamientos, lee (sin escribir) la colección reportes_mascotas para buscar coincidencias, y guarda las fotos en avistamientos/{uid}/{timestamp}.jpg dentro de Storage.

6. Diagramas UML — CU01: Gestionar Comentarios

A continuación explico, uno por uno, los cinco diagramas que elaboré para este caso de uso, tanto para la fase de análisis como para la fase de diseño, ya que —como expliqué antes— comparten el mismo contenido en ambas fases.

Figuras 01 y 06 — Diagrama de paquetes

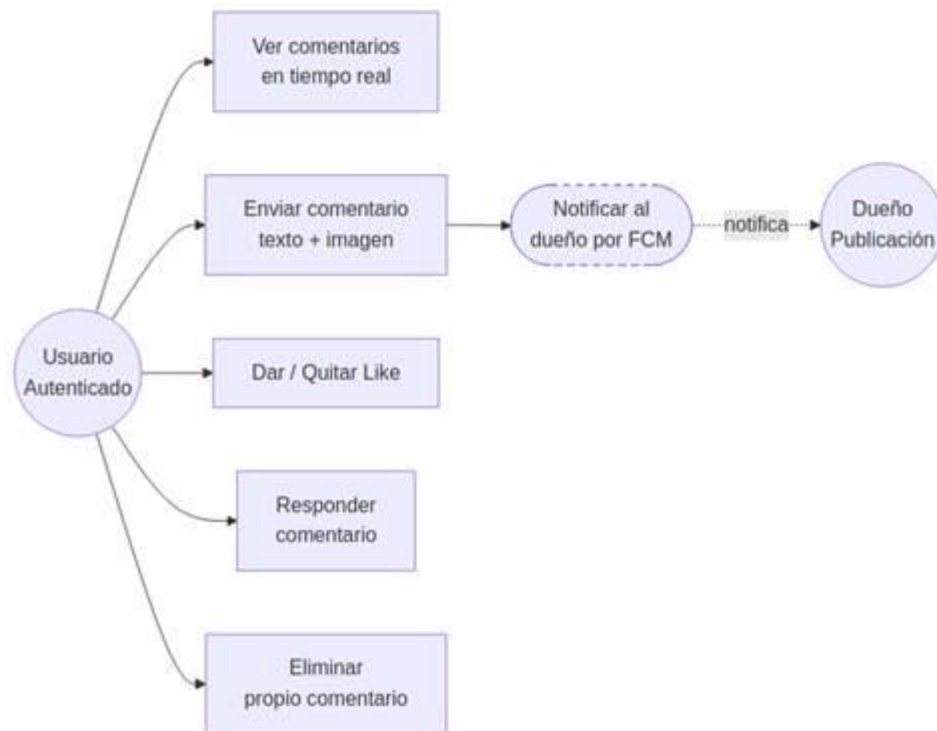


Figuras 01 (análisis) y 06 (diseño). Diagrama de paquetes del módulo *demo_comentarios*.

Este diagrama muestra cómo organicé las carpetas del proyecto y cómo se conectan entre ellas. Se ve la capa de Vista (*pantalla_comentarios.dart*), que le habla a la capa de ViewModel (*comentarios_viewmodel.dart*) y también a la de Utilidades (*fecha_utils.dart*) para dar formato a las fechas. El ViewModel, a su vez, se conecta con la capa de Modelo (*comentario_model.dart*) y con la infraestructura de Firebase, concretamente con Firestore. La

Vista también habla directamente con Firestore, con Storage (para subir imágenes) y con la capa de Servicios (notificacion_servicio.dart), la cual finalmente envía las notificaciones push por FCM. Dibujé este diagrama para dejar clara la separación de responsabilidades: cada flecha representa una dependencia real que existe en el código, ni una más ni una menos.

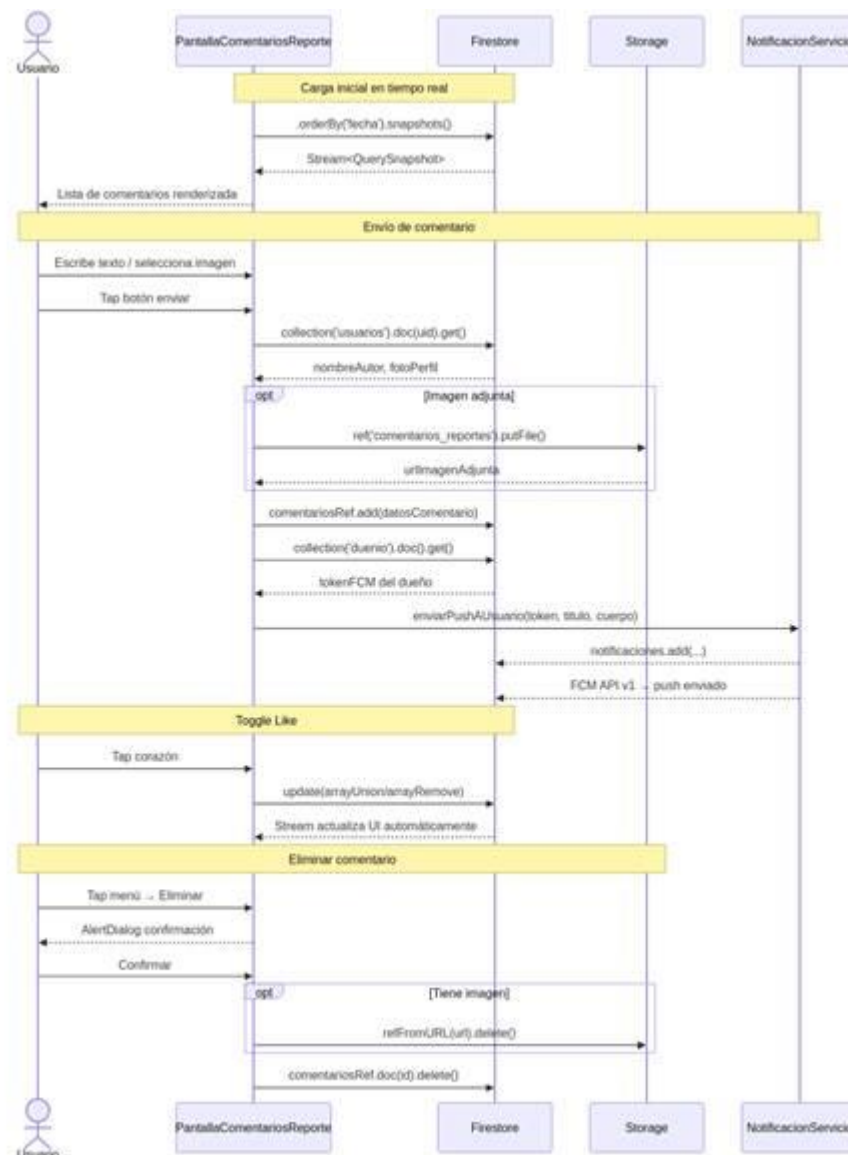
Figuras 02 y 07 — Diagrama de caso de uso



Figuras 02 (análisis) y 07 (diseño). Diagrama de caso de uso de Gestionar Comentarios.

Aquí muestro qué puede hacer el usuario autenticado dentro de este módulo: ver los comentarios en tiempo real, enviar un comentario con texto o imagen, dar o quitar “me gusta”, responder a un comentario y eliminar su propio comentario. Agregué también un caso de uso “Notificar al dueño por FCM”, dibujado con línea punteada porque no es una acción que el usuario ejecute directamente, sino una consecuencia automática de enviar un comentario: cuando alguien comenta, el sistema por su cuenta le avisa al dueño de la publicación. Por eso ese caso de uso está conectado con una flecha punteada hacia el actor “Dueño Publicación”, que es quien recibe el resultado de esa acción sin haberla iniciado.

Figuras 03 y 08 — Diagrama de secuencia

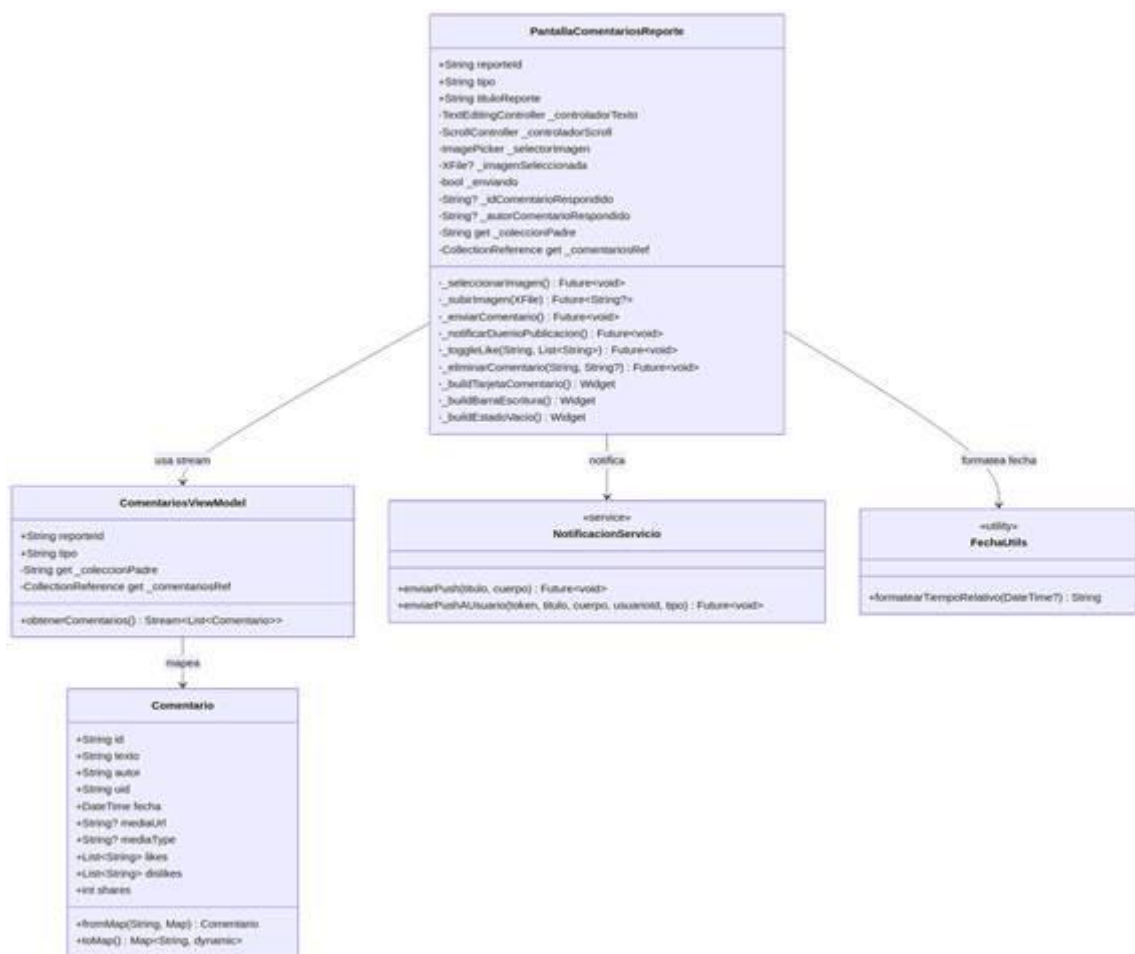


Figuras 03 (análisis) y 08 (diseño). Diagrama de secuencia de Gestionar Comentarios.

Este diagrama es el más largo porque muestra, paso a paso y en orden de tiempo, cuatro flujos distintos que pueden pasar en esta pantalla. El primero es la carga inicial: la vista le pide a Firestore el stream de comentarios ordenado por fecha, y Firestore le devuelve ese stream para que se muestre en pantalla. El segundo es el envío de un comentario: el usuario escribe texto o elige una imagen, la vista busca el nombre y la foto de perfil del usuario en Firestore, si hay imagen la sube a Storage y obtiene su URL, guarda el comentario en Firestore, busca el token de notificaciones del dueño de la publicación y le manda el mensaje push a través de

NotificacionServicio. El tercer flujo es el de dar “me gusta”: al tocar el corazón, se actualiza el arreglo de likes en Firestore usando arrayUnion o arrayRemove según corresponda, y el stream actualiza la pantalla automáticamente sin que yo tenga que refrescar nada a mano. El cuarto flujo es eliminar un comentario: aparece un cuadro de confirmación, y si el usuario confirma, primero se borra la imagen de Storage (si tenía una) y después se borra el documento del comentario en Firestore.

Figuras 04 y 09 — Diagrama de clases



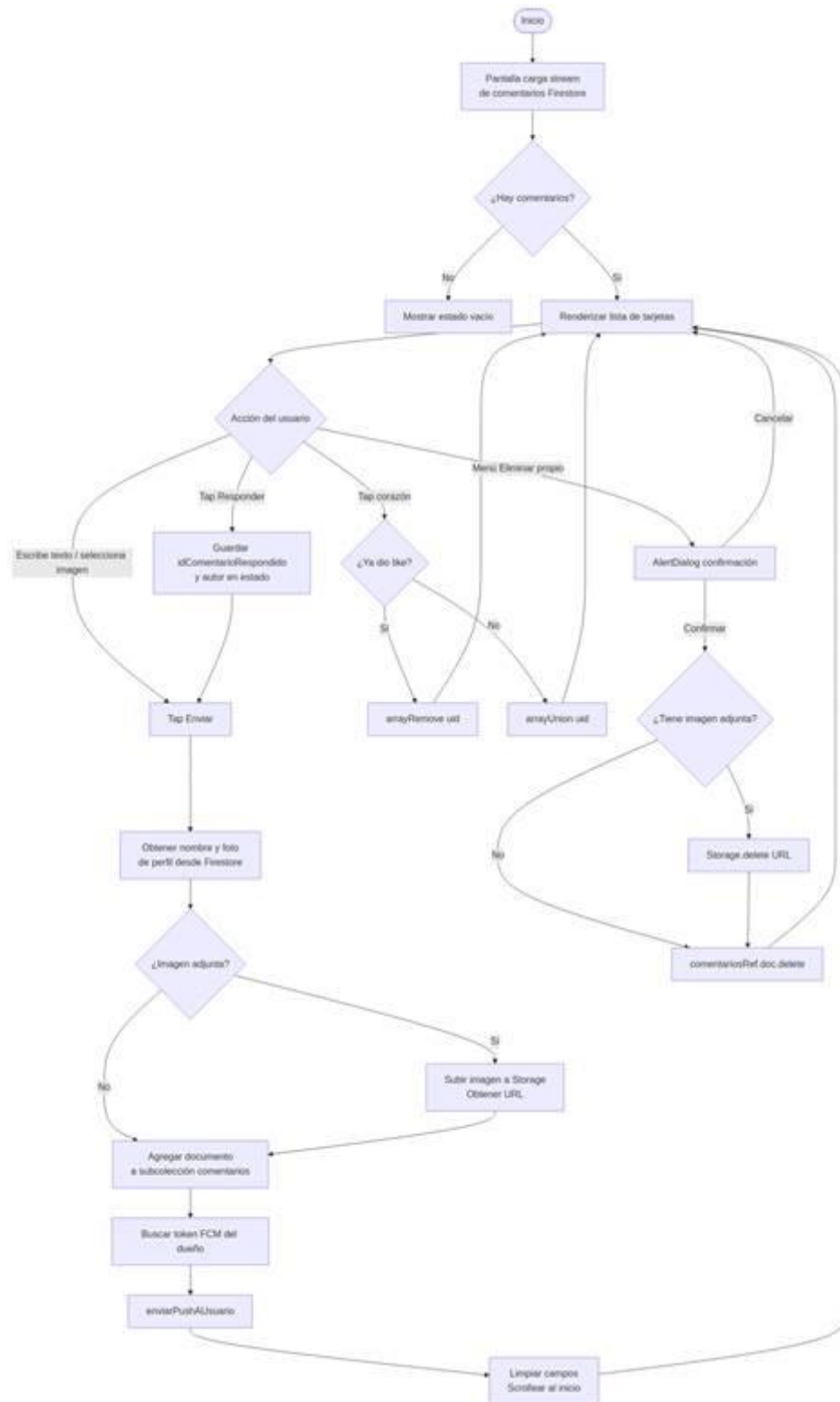
Figuras 04 (análisis) y 09 (diseño). Diagrama de clases de Gestionar Comentarios.

Este diagrama muestra las clases que forman el módulo y sus atributos y métodos principales. La clase central es **PantallaComentariosReporte**, que guarda el estado de la pantalla (el controlador del cuadro de texto, si se está enviando un comentario, a qué comentario se está



respondiendo, etc.) y tiene los métodos privados que ejecutan cada acción, como `_enviarComentario()` o `_toggleLike()`. Esta clase usa el stream que expone `ComentariosViewModel`, y ese `ViewModel` a su vez transforma cada documento de Firestore en un objeto `Comentario`, que es la clase que representa los datos (texto, autor, fecha, likes, dislikes, etc.) con sus métodos `fromMap()` y `toMap()` para convertir de y hacia el formato de Firestore. La pantalla también usa `NotificacionServicio` para notificar y `FechaUtils` para mostrar la fecha en un formato relativo, como “hace 5 minutos”.

Figuras 05 y 10 — Diagrama de actividades



Figuras 05 (análisis) y 10 (diseño). Diagrama de actividades de Gestionar Comentarios.

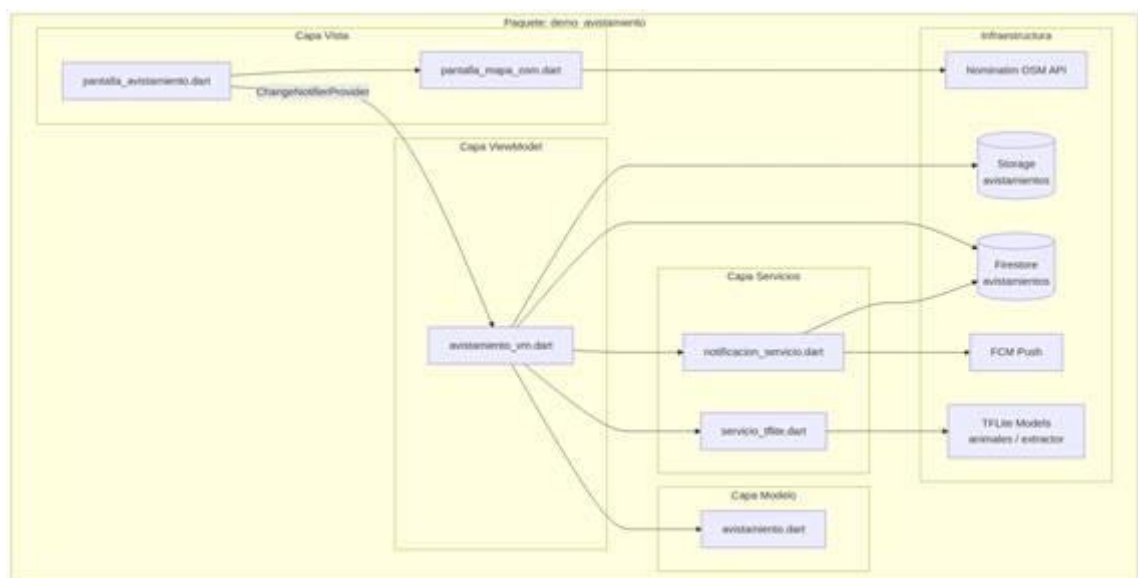
Este diagrama representa el flujo completo de decisiones que puede tomar el usuario

dentro de la pantalla, desde que se carga el stream de comentarios hasta las cuatro acciones posibles que ya mencioné en el diagrama de secuencia. Lo que aporta este diagrama en particular son las condiciones: por ejemplo, antes de enviar un comentario se pregunta si hay una imagen adjunta o no, y solo si la hay se ejecuta el paso de subirla a Storage; en el caso del “me gusta”, se pregunta si el usuario ya había dado like antes, para decidir si se debe agregar o quitar su ID de la lista; y en el caso de eliminar, se pregunta si el comentario tenía una imagen, para saber si hace falta borrar algo de Storage antes de borrar el documento. Dibujar estas decisiones ayuda a comprobar que el código maneja todos los caminos posibles y no se olvida de ningún caso especial.

11. Diagramas UML — CU02: Registrar Avistamiento

De la misma manera, explico a continuación los cinco diagramas correspondientes al segundo caso de uso, el de registrar un avistamiento.

Figuras 11 y 16 — Diagrama de paquetes

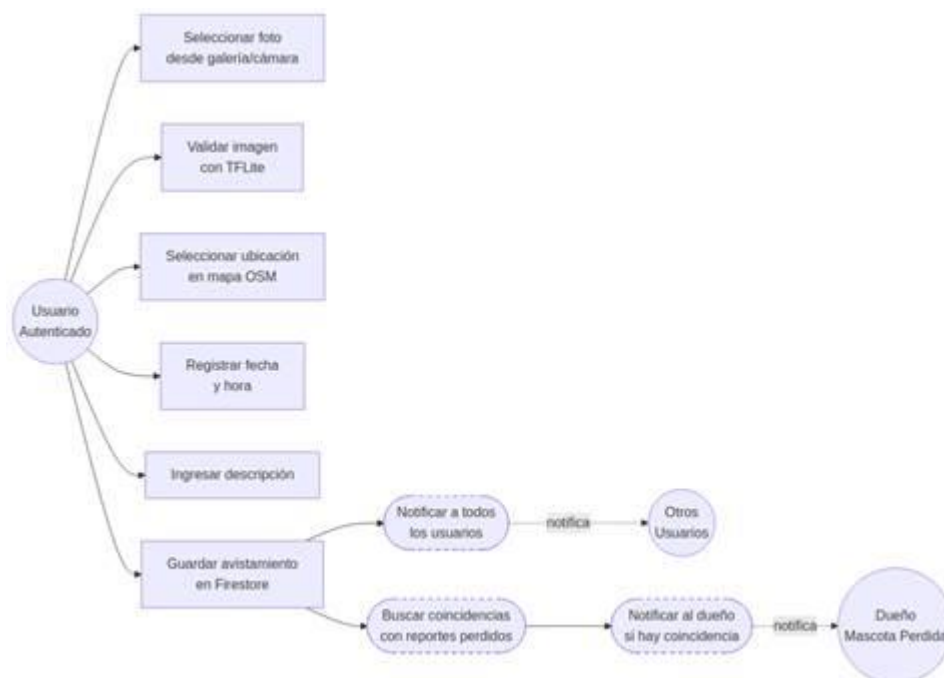


Figuras 11 (análisis) y 16 (diseño). Diagrama de paquetes del módulo *demo_avistamiento*.

Este módulo tiene una capa de Vista un poco más grande porque incluye dos pantallas:

pantalla_avistamiento.dart (el formulario) y pantalla_mapa_osm.dart (el mapa para elegir la ubicación). La vista del formulario se conecta con el ViewModel usando ChangeNotifierProvider, que es la forma en que Provider avisa a la pantalla cuando algo cambia en el ViewModel. El ViewModel (avistamiento_ym.dart) se conecta con el Modelo (avistamiento.dart), con el servicio de TFLite para detectar animales, con el servicio de notificaciones, y directamente con Firestore y Storage para guardar los datos y la foto. La pantalla del mapa, por su parte, se conecta con la API externa de Nominatim para convertir coordenadas en una dirección legible.

Figuras 12 y 17 — Diagrama de caso de uso

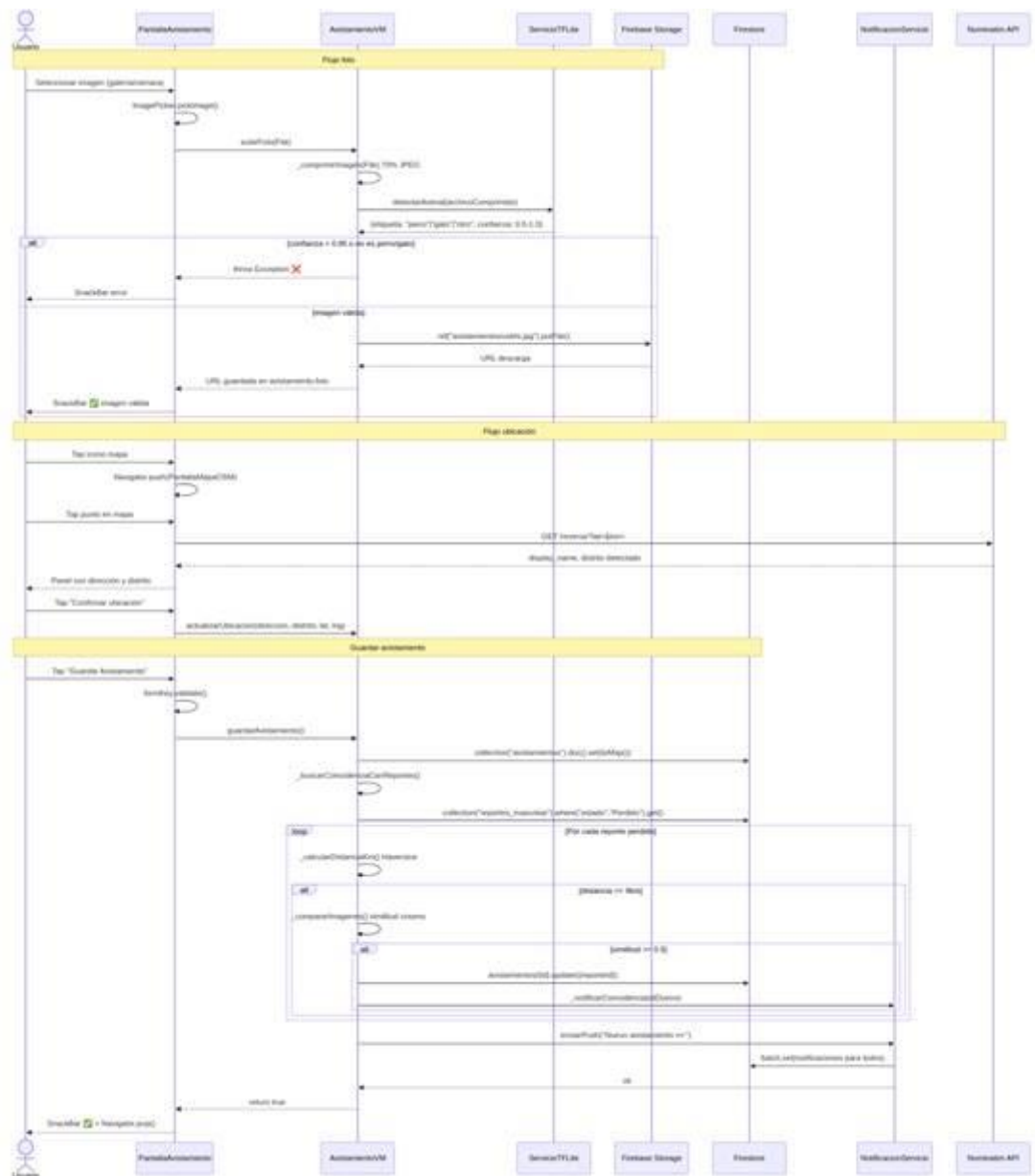


Figuras 12 (análisis) y 17 (diseño). Diagrama de caso de uso de Registrar Avistamiento.

Aquí se ven las acciones que el usuario autenticado puede realizar: seleccionar una foto desde la galería o la cámara, validar esa imagen con TFLite, elegir la ubicación en el mapa, registrar la fecha y hora, escribir una descripción y guardar el avistamiento. A partir de guardar el avistamiento se disparan tres cosas de forma automática (por eso están con línea punteada): el sistema busca coincidencias con reportes de mascotas perdidas, notifica a todos los usuarios de la aplicación que hay un nuevo avistamiento, y si encontró una coincidencia, notifica

específicamente al dueño de la mascota perdida. Por eso en el diagrama aparecen dos actores adicionales aparte del usuario: “Otros Usuarios”, que reciben la notificación general, y “Dueño Mascota Perdida”, que recibe la notificación de coincidencia.

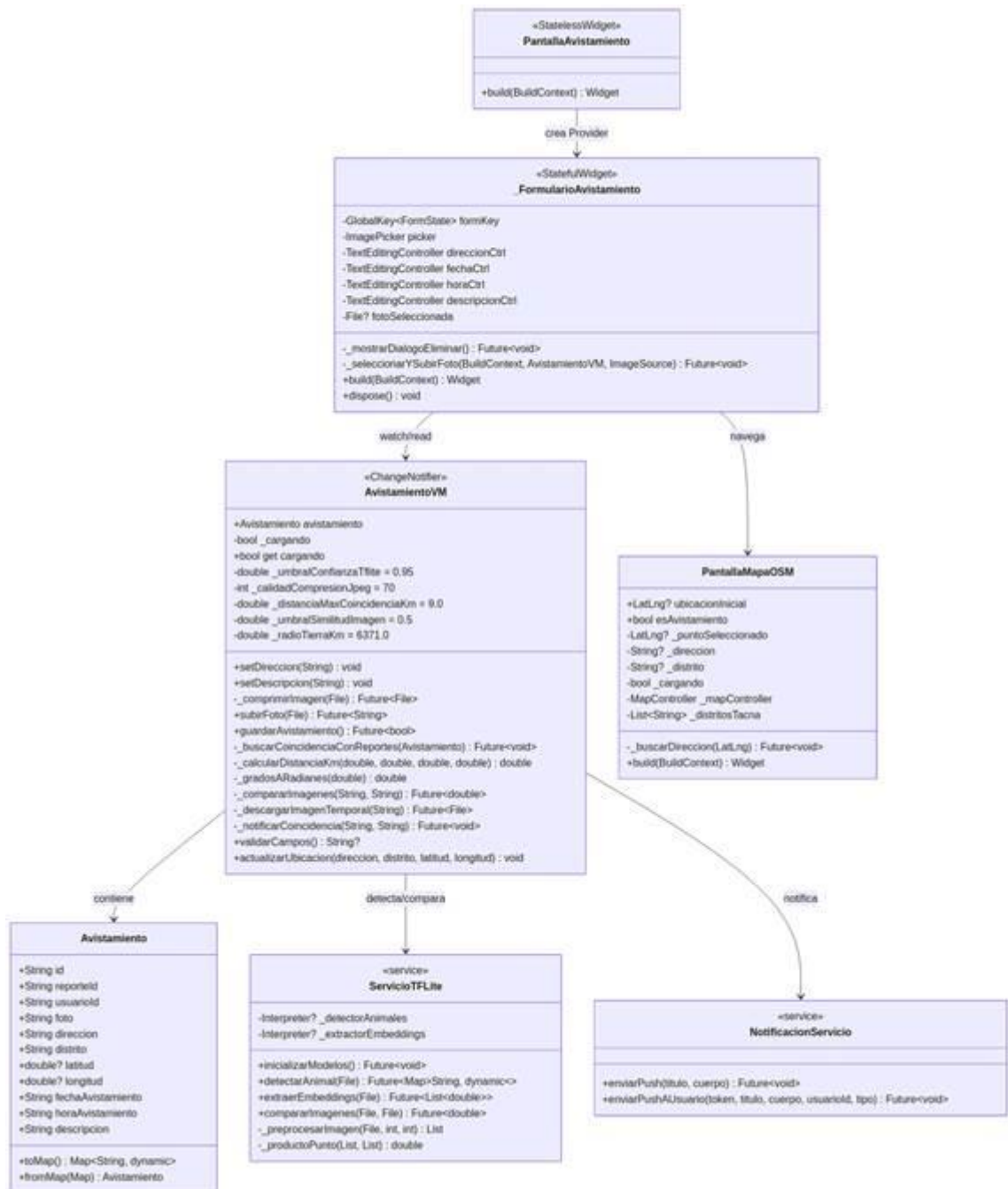
Figuras 13 y 18 — Diagrama de secuencia



Figuras 13 (análisis) y 18 (diseño). Diagrama de secuencia de Registrar Avistamiento.

Divido este diagrama en tres partes. La primera es el flujo de la foto: el usuario elige una imagen,

la vista se la pasa al ViewModel, el ViewModel la comprime al 70 % de calidad JPEG para que no pese demasiado y se la envía al servicio de TFLite, el cual devuelve una etiqueta (“perro”, “gato” u “otro”) junto con un porcentaje de confianza. Si la confianza es menor a 0.95 o el animal detectado no es un perro o un gato, se lanza un error y se le muestra un aviso al usuario; si la imagen es válida, recién ahí se sube a Firebase Storage y se guarda la URL. La segunda parte es el flujo de ubicación: el usuario toca un punto en el mapa, la app le hace una consulta a la API de Nominatim para convertir esas coordenadas en una dirección y un distrito, se lo muestra al usuario, y cuando confirma, esos datos se guardan en el ViewModel. La tercera parte es guardar el avistamiento: se valida el formulario, se guarda el documento en Firestore, y automáticamente se recorren los reportes de mascotas perdidas calculando la distancia con la fórmula de Haversine; si algún reporte está a 9 km o menos, se comparan las imágenes por similitud de coseno, y si esa similitud es de 0.5 o más, se vincula el avistamiento con ese reporte y se notifica al dueño. Al final, sin importar si hubo o no coincidencia, se envía una notificación general a todos los usuarios avisando que hay un nuevo avistamiento.



Figuras 14 (análisis) y 19 (diseño). Diagrama de clases de Registrar Avistamiento.

Este es el diagrama de clases más completo del proyecto, porque el módulo de avistamiento tiene más lógica que el de comentarios. Se ve la clase PantallaAvistamiento, que crea el Provider y muestra el formulario (_FormularioAvistamiento), la cual mantiene los cuatro controladores de texto que mencioné en la sección de calidad de código, además del método dispose() que agregué para liberarlos. Esa pantalla usa AvistamientoVM, la clase más grande del

diagrama, que guarda las constantes de configuración del negocio como `_umbralConfianzaTflite` (0.95), `_distanciaMaxCoincidenciaKm` (9.0) y `_umbralSimilitudImagen` (0.5), y que tiene métodos como `guardarAvistamiento()`, `_buscarCoincidenciaConReportes()` y `_calcularDistanciaKm()`. Esta clase se conecta con `Avistamiento` (el modelo de datos), con `ServicioTFLite` (que hace la detección y comparación de imágenes) y con `NotificacionServicio`. También se muestra `PantallaMapaOSM` con su parámetro `esAvistamiento`, que es justamente el que agregué como parte de la mejora de calidad explicada en la sección 6.2.

Figuras 15 y 20 — Diagrama de actividades

Este diagrama es demasiado extenso para mostrarlo en una sola imagen legible, así que lo dividí en dos partes que se continúan una a la otra.

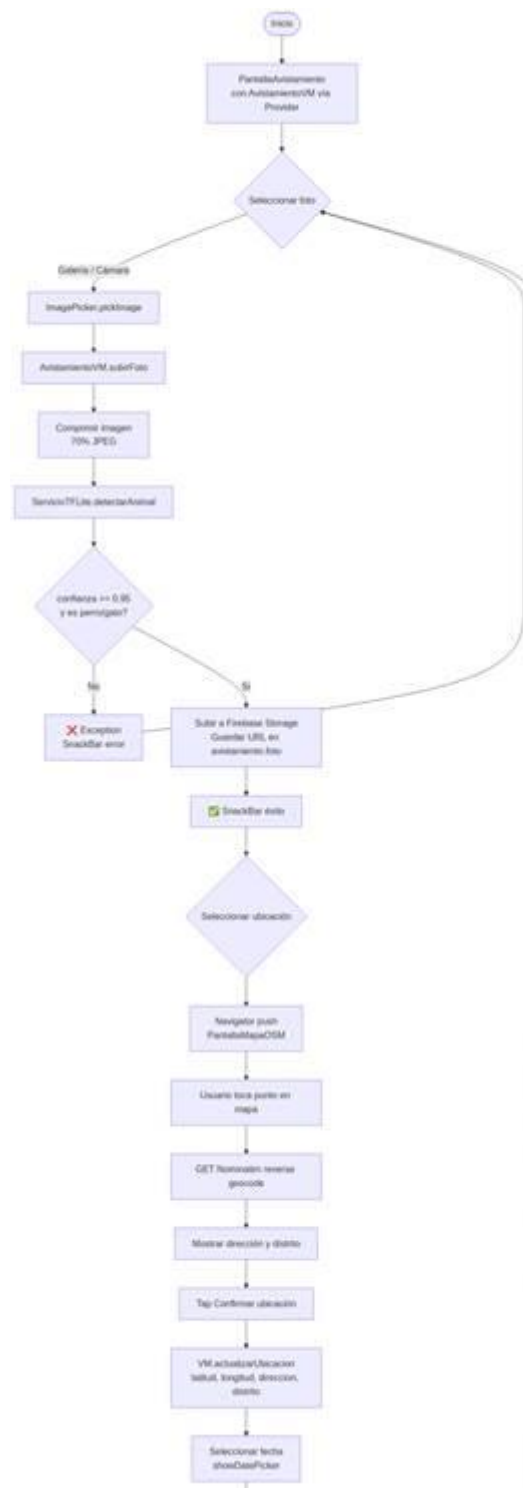


Figura 15/20 (parte 1 de 2). Diagrama de actividades de Registrar Avistamiento: desde la selección de la foto hasta la confirmación de la ubicación.

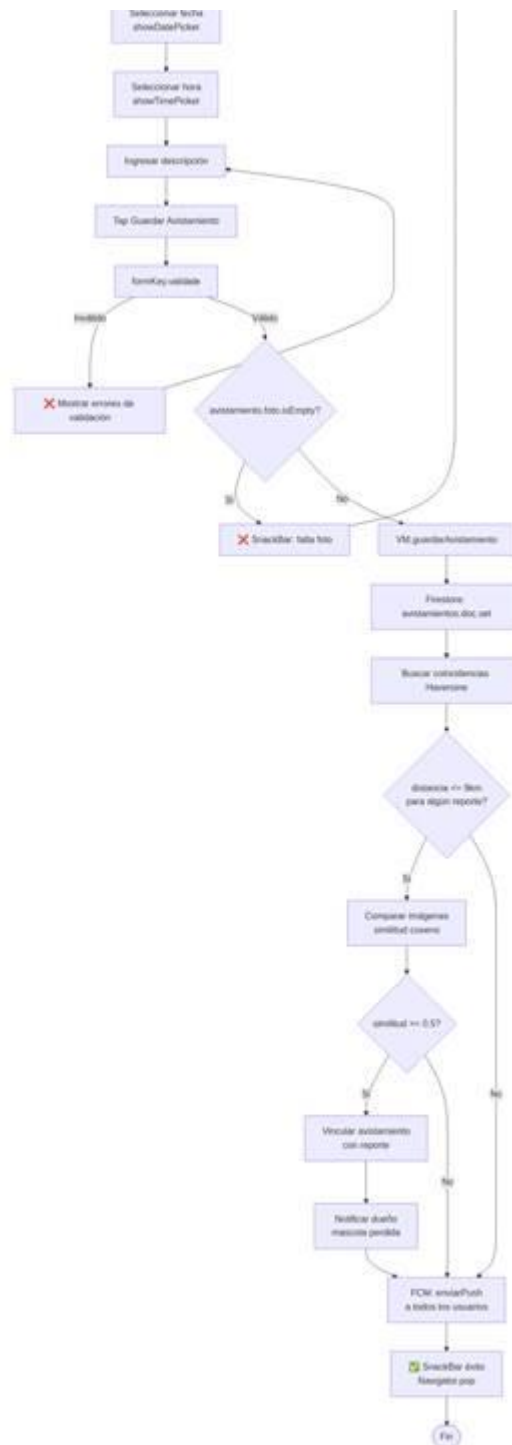


Figura 15/20 (parte 2 de 2). Diagrama de actividades de Registrar Avistamiento: desde el ingreso de fecha y descripción hasta el guardado final y las notificaciones.

En la primera parte del diagrama se ve el flujo de la foto: el usuario la selecciona, se comprime, se manda a TFLite, y hay una decisión clave que pregunta si la confianza es de al

menos 0.95 y si el animal detectado es un perro o un gato; si la respuesta es no, se muestra un error y se regresa a pedir que seleccione otra foto; si la respuesta es sí, se sube la imagen y se avisa que fue aceptada. Después sigue el flujo de ubicación: se abre el mapa, el usuario toca un punto, se hace la geocodificación inversa con Nominatim, se muestra la dirección y el distrito, y se confirma la ubicación.

En la segunda parte se completa el formulario con la fecha, la hora y la descripción. Al tocar “Guardar Avistamiento” se valida el formulario; si algo está mal se muestran los errores y se regresa a completar los datos. Si el formulario es válido, se revisa si falta la foto (por si el usuario nunca llegó a subir una), y si falta, se le pide que la seleccione. Si todo está completo, se guarda el avistamiento en Firestore y se ejecuta la búsqueda de coincidencias con la fórmula de Haversine: si hay un reporte a 9 km o menos, se comparan las imágenes por similitud; si la similitud es de 0.5 o más, se vincula el avistamiento con el reporte y se notifica al dueño de la mascota perdida. Haya habido o no coincidencia, al final siempre se envía una notificación general a todos los usuarios y se le muestra al usuario un mensaje de éxito antes de regresar a la pantalla anterior.

7. Conclusiones

- El problema de calidad más importante que encontré en todo el proyecto fue la falta del método `dispose()` en `pantalla_avistamiento.dart`, que dejaba cuatro controladores de texto sin liberar de la memoria; corregirlo fue la mejora más significativa que apliqué en todo este trabajo.
- Encontré también código que ya no cumplía ningún propósito, como la función `_runNetworkDependentTasks()` en `main.dart`, la cual dependía de un flujo de login que las demos ya no tienen; eliminarla dejó el punto de entrada de la aplicación más limpio y fácil de entender.
- La mayoría de los archivos del proyecto original ya tenían una lógica correcta y bien separada por capas (vista, vistamodelo, modelo, servicios); el problema principal no era la lógica en sí, sino que las rutas de import usaban el nombre fijo del paquete `package:sos_mascotas/...`, lo que impedía que estos módulos funcionaran de forma independiente. Cambiarlas por rutas relativas fue necesario en casi todos los archivos revisados.

- Desacoplar `notificacion_servicio.dart` de la clase `Notificacion` y hacer que trabaje con `Map<String, dynamic>` demuestra que un servicio puede ser más flexible y reutilizable sin perder nada de su funcionalidad original.
- Verifiqué que ningún cambio de estructura afectara las reglas de negocio del proyecto: el umbral de 95 % de confianza en TFLite, la distancia máxima de 9 km para buscar coincidencias, el umbral de 0.5 de similitud entre imágenes y el envío de notificaciones por FCM siguen funcionando exactamente igual que en el proyecto principal.
- La matriz de trazabilidad y los diez diagramas UML me permitieron confirmar que cada clase y cada archivo de código que programé responde a una necesidad identificada desde la fase de análisis, sin que sobre ni falte ningún componente.

8. Recomendaciones

- Recomiendo revisar el resto de pantallas del proyecto principal `SosMascota` buscando el mismo problema de controladores sin `dispose()`, ya que es un error fácil de repetir sin darse cuenta en Flutter.
- Sugiero evitar en el futuro el uso de rutas de `import` con el nombre fijo del paquete cuando se sabe que un módulo podría necesitar separarse más adelante; usar rutas relativas desde un inicio habría evitado tener que corregir tantos archivos.
- Recomiendo mantener la costumbre de dejar comentarios de tipo `// CAMBIO` cuando se modifica código heredado de otra versión, porque facilita mucho volver a entender, meses después, por qué una línea es distinta a la original.
- Antes de eliminar una función que parece no usarse, sugiero primero comprobar con una búsqueda en todo el proyecto que en verdad no la llama ningún otro archivo, para no borrar por error algo que sí se necesitaba.

9. Referencias



- Martin, R. C. (2008). Clean code: A handbook of agile software craftsmanship. Prentice Hall.
- McConnell, S. (2004). Code complete: A practical handbook of software construction (2.^a ed.). Microsoft Press.
- Google. (2024). Flutter documentation. <https://docs.flutter.dev>
- Google. (2024). Firebase documentation. <https://firebase.google.com/docs>
- Taco Coayla, R. A. (2026). Evidencia de trazabilidad y cambios de código (antes vs. después): Proyecto SOS Mascota [Documento de curso]. Universidad Privada de Tacna, Escuela Profesional de Ingeniería de Sistemas.
- Taco Coayla, R. A. (2026). Matriz de trazabilidad SRS–SAD–Implementación: Proyecto SOS Mascota [Documento de curso]. Universidad Privada de Tacna, Escuela Profesional de Ingeniería de Sistemas.